Software Design Specification
for
PayMate

Version <0.1>

Prepared by

Group Name: Thursday

Chayathon Chaiyasak	**6522771300**	**6522771300@g.siit.tu.ac.th**
Aekkarin Yimyaem	**6522772498**	**6522772498@g.siit.tu.ac.th**
Kitssada Pavasutthipand	**6522772530**	**6522772530@g.siit.tu.ac.th**
Nattapol Phanmadee	**6522781994**	**6522781994@g.siit.tu.ac.th**
Pattarapong Tiemtan	**6522780897**	**6522780897@g.siit.tu.ac.th**

Instructor: Asst.Prof.Dr. Sasiporn Usanavasin

Course: Software Engineering

Date: May 13, 2025

1 Introduction....3
1.1 Document Purpose....3
1.2 Product Scope....3
1.3 Intended Audience and Document Overview....3
1.4 Definitions, Acronyms and Abbreviations....4
1.5 Document Conventions....4
1.6 References and Acknowledgments....5
2 Overall System Design....5
2.1 Architectural Design....5
2.2 Hardware Interface Design....9
2.3 Software Interface Design....9
2.4 Data Design....10
2.5 Design and Implementation Constraints....15
2.6 Assumptions and Dependencies....16
3 Detail Design Specification....18
3.1 Use Case Diagram for Functional Requirements....18
3.2 Class Diagram Design (OOP design)....19
4 Human Interface Design....39
4.1 System Navigation Design....39
4.2 Screen Design....40
5 Requirement Traceability....52
Appendix A – Data Dictionary....57

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
0.1	Nattapol Phanmadee, Aekkarin Yimyaem, Kitssada Pavasutthipand, Pattarapong Tiemtan, Chayathon Chaiyasak	Completed all the report	11/05/2025

1 Introduction

1.1 Document Purpose

This section provides a comprehensive introduction to the Software Design Specifications (SDS) for the project. It outlines the purpose and objectives of the system, highlights its overall scope, and identifies the intended audience of this document. The section also introduces key definitions, acronyms, and abbreviations used throughout the SDS to ensure clarity and consistency.

The primary goal of this document is to detail the functional and non-functional requirements of the system, serving as a foundation for system design, development, and testing. It establishes a shared understanding among stakeholders, including developers, project managers, quality assurance teams, and clients, ensuring all parties are aligned on what the system will deliver.

In the following sections, readers will find descriptions of the system's intended features, user interactions, performance expectations, constraints, and external interfaces. This SDS serves as a reference point throughout the software development lifecycle and supports traceability from requirements to implementation.

1.2 Product Scope

PayMate is a comprehensive receipt scanner and expense analyzer designed to simplify personal finance management. Its primary purpose is to help users efficiently track, categorize, and optimize their expenses by leveraging advanced technologies such as Optical Character Recognition (OCR) for receipt scanning and machine learning for expense categorization. PayMate aims to provide users with a clear overview of their spending habits, enabling them to make informed financial decisions. The application offers features like bill splitting, budget tracking, and cost-cutting suggestions, making it an indispensable tool for individuals seeking to manage their finances effectively.

The benefits of using PayMate include enhanced financial transparency, reduced time spent on manual expense tracking, and improved budget adherence. By automatically categorizing expenses into predefined categories and providing insights into spending trends, PayMate helps users identify areas for cost reduction and optimize their financial resources. Additionally, features such as group expense logging and shared budget tracking facilitate collaborative financial management among friends or family members. Overall, PayMate's objectives are to streamline financial management, promote fiscal responsibility, and offer users a user-friendly interface to achieve their financial goals.

1.3 Intended Audience and Document Overview

PayMate is primarily intended for two key audiences:

1. **Client:** This document serves as a comprehensive representation of PayMate's intended functionality and requirements. The client can review this SRS to ensure the application aligns with their vision and expectations, verifying that all essential features and aspects are adequately addressed.
2. **Professor:** This SRS provides detailed information about the project’s objectives, scope, and technical implementation. The professor can use this document to evaluate the project's adherence to software engineering principles and assess the students' understanding of requirements analysis and specification.

To facilitate a clear understanding of PayMate, this SRS is structured into the following sections:

1. **Introduction:** Provides an overview of the project, its purpose, and the intended audience of this document. It establishes the context for understanding the subsequent detailed requirements.
2. **Overall Description:** Describes the general context, functionality, constraints, and assumptions related to PayMate. This section offers a high-level perspective on the product and its interactions with the environment.
3. **Specific Requirements:** Details the specific business and user requirements, external interface requirements, functional requirements, and use case model for PayMate. This section is crucial for understanding the precise functionalities and user interactions supported by the system.
4. **Other Non-Functional Requirements:** Specifies the non-functional requirements, including performance, safety, security, and software quality attributes. This ensures that PayMate meets the necessary standards for usability, reliability, and security.
5. **Requirements Traceability:** This will link user requirements to test cases.
6. **Appendix A** – Data Dictionary: Provides a comprehensive list of data elements used within PayMate, along with their definitions, types, and constraints.
7. **Appendix B** - Group Log: Records the activities and contributions of the team members during the development of this SRS.

1.4 Definitions, Acronyms and Abbreviations

Term/Acronym	Description
API	Application Programming Interface
COMET	Concurrent Object-Oriented Modeling and Architectural Design
OCR	Optical Character Recognition
SRS	Software Requirements Specification
FR	Functional Requirement
TLS	Transport Layer Security
FR	Functional Requirement
UML	Unified Modeling Language

1.5 Document Conventions

Formatting Conventions:

- *Arial font, size 11 for regular text.*
- *Italics for comments and explanations.*
- *Bold for titles and important terms.*

Naming Conventions:

- *Functional Requirements: FR001, FR002, etc.*
- *Use Cases: UC001, UC002, etc.*
- *User Requirements: USR001, USR002, etc.*

1.6 References and Acknowledgments

PayMate version 0.1
https://docs.google.com/document/d/18MRryN_amwAXMvieMwbksPfCdPUVRfaNBeeFTF_h5ek/edit?usp=sharing

2 Overall System Design

2.1 Architectural Design

The deployment diagram presented illustrates the architectural design of the PayMate system, a mobile-centric financial application that facilitates receipt management, transaction tracking, and user interaction with banking data. This architecture comprises multiple interconnected subsystems, each responsible for specific functionalities, and is structured around four main environments: the Smartphone (mobile client), the PayMate Database Server, the Bank Server, and a Cloud Server.

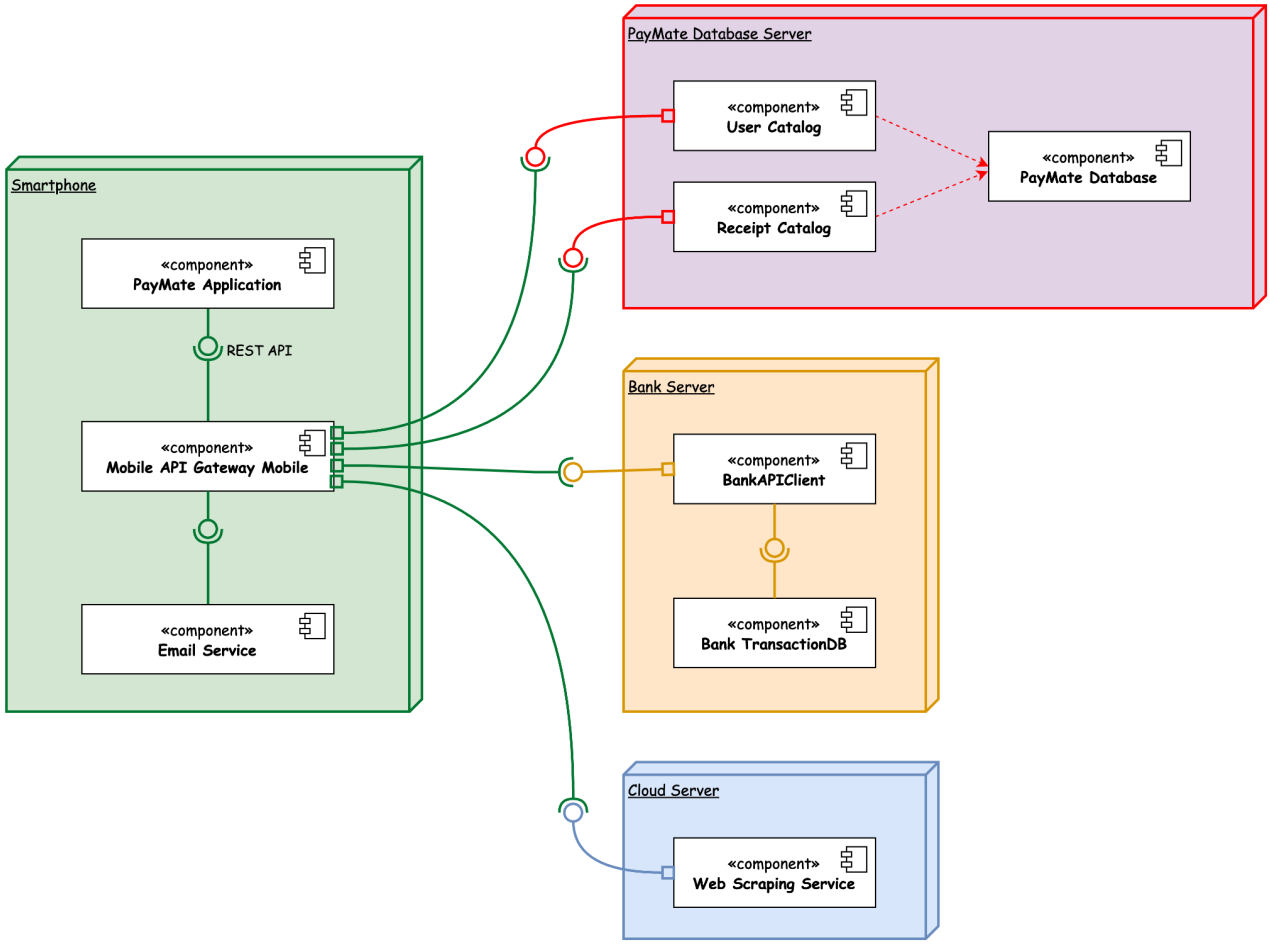


Figure 5.1.2 Deployment Diagram
https://drive.google.com/file/d/1Qbh3seuxSU_F0S4869KtdfCqU2pOzCz_/view?usp=drive_link

Mobile Client Components (Smartphone)

At the core of the user experience lies the Smartphone, housing the user-facing PayMate Application. This component serves as the interface through which users interact with the system—scanning receipts, viewing transactions, and managing account information. All interactions from the application are routed through the Mobile API Gateway Mobile, a critical component acting as the mediator between the mobile app and backend services. This gateway ensures secure and standardized communication via REST APIs, and orchestrates data exchange across the system.

Additionally, the Email Service component within the mobile environment plays a vital role in user notifications, such as verification emails, receipt confirmations, and system alerts. This service ensures that user communications are automated and reliable, enhancing overall engagement and trust in the platform.

PayMate Database Server

The PayMate Database Server hosts essential backend components that store and manage data generated by users. Two primary catalogs reside in this environment:

- The User Catalog handles all user-related data, including authentication credentials, preferences, and profiles.
- The Receipt Catalog manages data extracted from scanned or uploaded receipts, supporting the application's core functionality of expense tracking and categorization.

Both of these catalogs interact with a shared PayMate Database, which serves as the centralized data repository. The diagram uses dashed arrows to represent this dependency, signifying that both catalogs rely on database operations for persistent storage and retrieval. Communication between the mobile gateway and these backend components is shown via green connectors, reflecting API interactions over the network.

Bank Server

The Bank Server is responsible for facilitating secure access to users' financial transaction data. It comprises two key components:

- The BankAPIClient connects to external banking APIs, enabling real-time retrieval of account and transaction details from partnered financial institutions.
- The Bank TransactionDB stores the fetched banking data, which is later used for analysis, visualization, or budgeting within the PayMate application.

This structure supports a modular integration with banking systems, ensuring that financial data is synchronized and up-to-date while maintaining separation of concerns through distinct client and storage components.

Cloud Server

Contrary to the earlier label in the diagram, the blue-shaded environment represents a Cloud Server, not a bank server. This Cloud Server hosts the Web Scraping Service, an auxiliary component designed to extract transaction data from bank websites when official APIs are not available. This component demonstrates the system's adaptability, providing a fallback mechanism to access user data from less-integrated banking sources.

The scraping service communicates with the Mobile API Gateway to deliver extracted data into the same pipeline used by API-based sources, ensuring uniformity in the data processing workflow.

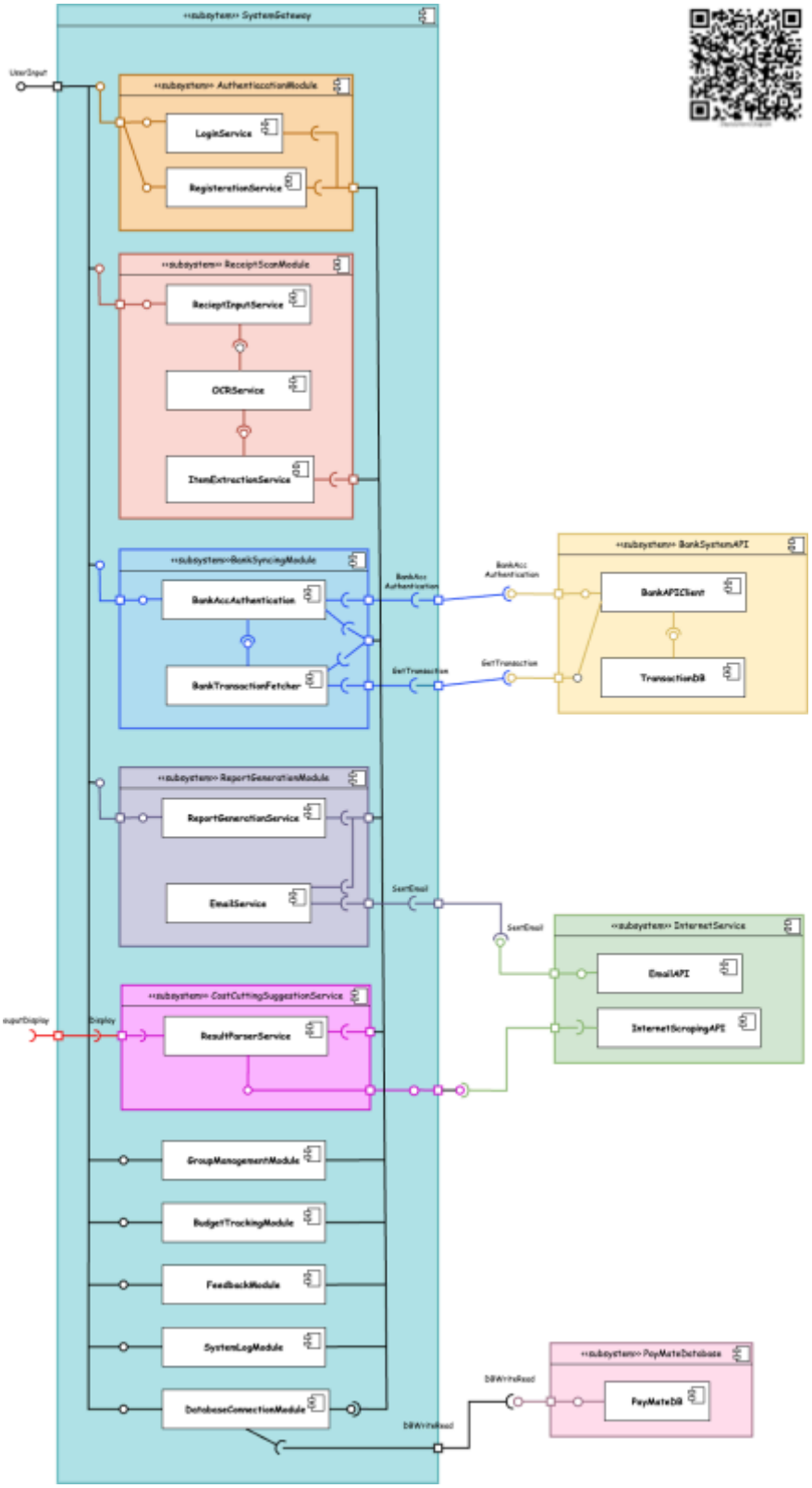


Figure 5.1.2 Component Diagram
https://drive.google.com/file/d/1PmT9wG6DgEo5i2TKm-6zunhvbtWG__rR/view?usp=drive_link

2.2 Hardware Interface Design

This section outlines the hardware interface designs that allows the interaction between the software and the physical inputs. The software will interact with the following hardware components:

Mobile Device (Smartphone/Tablet): The app will be installed and run on user mobile devices (iOS and Android). The device will be used for scanning receipts, managing expenses, syncing bank data, and performing other core app functionalities.
Camera: The app will interact with the mobile device's camera to capture images of receipts. It will then use OCR (Optical Character Recognition) services (either on-device or in the cloud) to extract text and item details from the receipt images.
Network Interface Components: The app will use the mobile device's network capabilities (Wi-Fi, cellular data) to send receipt images and related data to the cloud for processing and analysis. It will also connect to external APIs for tasks like web scraping or syncing bank data.

2.3 Software Interface Design

This section outlines the interfaces between the various software components, libraries, APIs, and external services used by the system. It describes how different modules of the application interact externally with third-party software/API systems to achieve full functionality.
Internal Software Interfaces:

- **Frontend ↔ Backend API**
  - **Interface:** RESTful API
  - **Protocol:** HTTP/HTTPS
  - **Format:** JSON
  - **Functionality:** Handles login, receipt upload, data fetching, report generation

Third-Party API Interfaces:

API/Library	Purpose	Type	Data Format	Protocol
Google Vision API	OCR for receipts	REST API	JSON	HTTPS
Firebase Auth	User login/registration	SDK + REST API	JWT	HTTPS
Plaid API	Bank transaction sync	REST API	JSON	HTTPS
SendGrid API	Sending reports via email	REST API	JSON	HTTPS

Database Interface:

- **Type:** MySQL
- **Access Method:** direct SQL queries
- **Operations:** CRUD
- **Data Format:** Structured (SQL tables)

2.4 Data Design

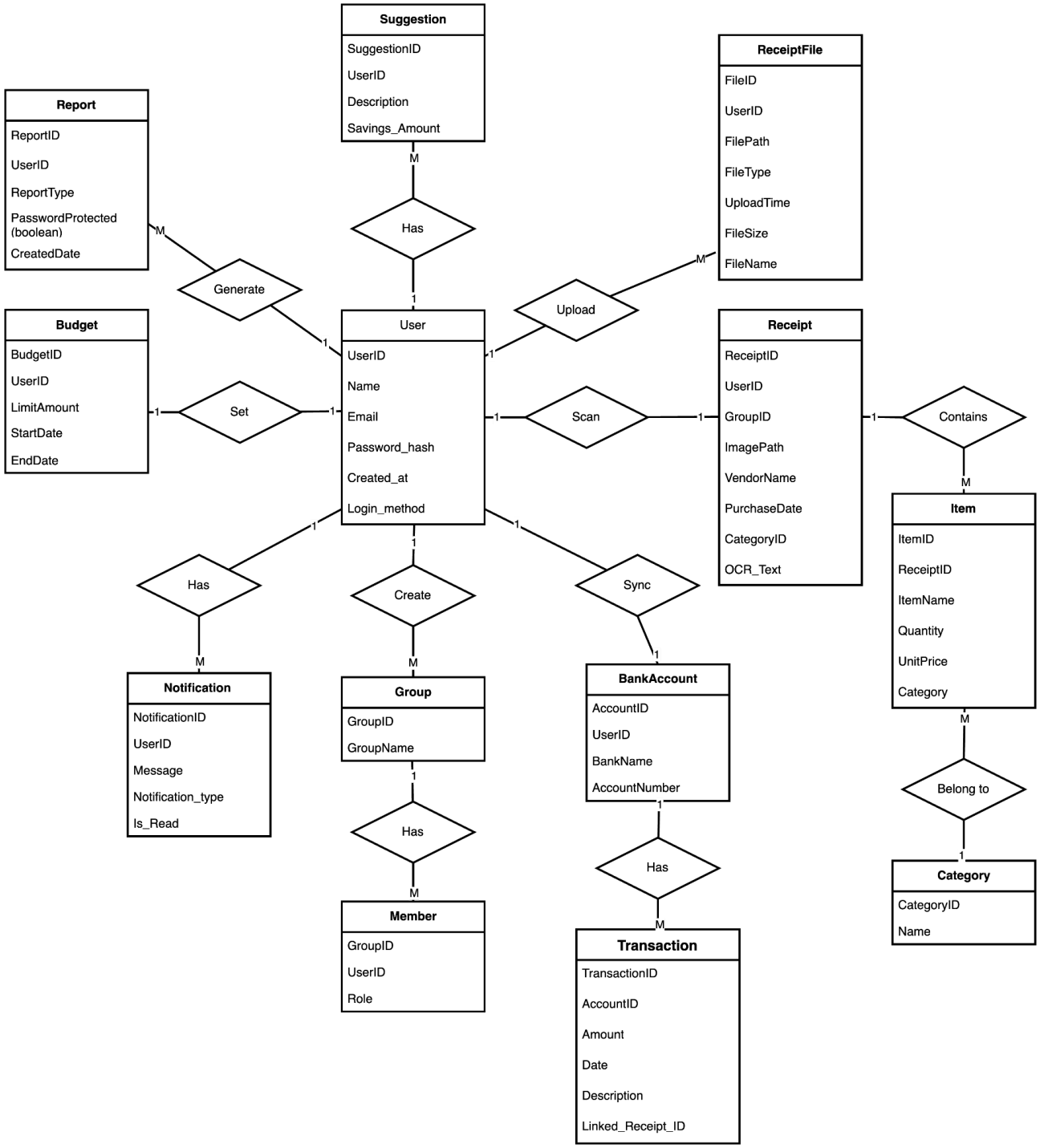


Figure 2.4.2.2 ERD diagram

2.4.1 Data Description

The information domain of the PayMate system has been transformed into structured data models that represent users, expenses, receipts, budgets, reports, and group activities. Each of these entities has been carefully modeled to support the core functionalities of expense tracking, utility bill recognition, cost-saving suggestions, and group-based financial management.

Major Entities and Data Structures:

1. **User**
   - Fields: UserID, Name, Email, Password_hash, Created_at
   - *Purpose:* Represents a registered user in the system.
   - Storage: Stored in a Users table in a relational database (e.g., PostgreSQL).

2. **Receipt**
   - Fields: ReceiptID, UserID, GroupID, ImagePath, VendorName, PurchaseDate, CategoryID, OCR_Text
   - Purpose: Represents uploaded receipts or bills; OCR extracts and parses data for further use.
   - Storage: Receipt images are stored in cloud storage ; metadata is stored in a Receipts table.

3. **Budget**
   - Fields: BudgetID, UserID, LimitAmount, StartDate, EndDate
   - Purpose: Allows users to define and track spending limits per category.
   - Storage: Stored in a Budgets table.

4. **Group**
   - Fields: GroupID and GroupName
   - Purpose: Supports household or shared expense tracking.
   - Storage: Stored in Groups and GroupMembers tables.

5. **Reports**
   - Fields: ReportID, UserID, ReportType, PasswordProtected, CreatedDate
   - Purpose: Stores metadata for periodic reports generated in PDF, PNG, or CSV formats.
   - Storage: Files stored in cloud storage; metadata stored in a Reports table.

6. **Suggestions**
   - Fields: SuggestionID, UserID, Description, Savings_Amount
   - Purpose: AI-generated insights or recommendations based on historical spending.
   - Storage: Stored in a Suggestions table.

7. **Notification**
   - Fields: NotificationID, UserID, Message, Notification_type, Is_Read

- Purpose: Represents a system-generated message sent to a user, alerting them of important events such as bill due dates, budget alerts, report completion, or group activity.
- Storage: Stored in a Notifications table in a relational database.

8. **Bank Account**
   - Fields: AccountID, UserID, BankName, AccountNumber
   - Purpose: Represents a user's linked bank account used for syncing transaction data into the system for automatic expense tracking and analysis.
   - Storage: Stored in a BankAccounts table in a relational database. Sensitive data such as full account numbers or tokens is encrypted or tokenized, and only necessary metadata is stored in plain form for display.

Data Processing Overview:

- *OCR Pipeline:* Images uploaded are sent to an OCR engine (e.g., Tesseract or AWS Textract), which returns raw text. This text is then parsed and mapped to structured fields (vendor, date, total, due date).
- *Bank Sync:* Uses APIs or secure file imports to sync transaction data into the system.
- *Cost Analysis :* Uses pre-processed expenses to identify overspending patterns and recommend savings.
- *Group Sync :* Updates shared expense records in real-time for all group members.

Databases and Storage:

- *Relational Database:* PostgreSQL or MySQL used for structured data (users, receipts, expenses, budgets).
- *Cloud File Storage:* Images and report files stored using services like Amazon S3 or Firebase.
- *In-Memory Cache:* Redis used for session handling and quick retrieval of frequently accessed data (e.g., budgets, recent expenses).
- Optional NoSQL Store: MongoDB or similar may be used for storing OCR text or logs for flexible access.

2.4.2 Data Dictionary

Entity	Field Name	Data Type	Description
User	UserID	INT	Primary Key
	Name	VARCHAR	Full name of the user
	Email	VARCHAR	Unique email address
	Password_hash	VARCHAR	Hashed password
	Created_at	DATETIME	Account creation timestamp
	Login_method	VARCHAR	"email", "google", "apple", etc.
Receipt	ReceiptID	INT	Primary Key
	UserID	INT	Foreign Key → User.UserID
	GroupID	INT	Foreign Key → Group.GroupID(nullable)
	ImagePath	VARCHAR	Path to the receipt image
	VendorName	VARCHAR	Vendor/store name
	PurchaseDate	DATE	Date of the transaction
	CategoryID	INT	Foreign Key → Category.CategoryID
	OCR_Text	TEXT	Raw OCR output for future validation
Category	CategoryID	INT	Primary Key
	Name	VARCHAR	E.g., Electricity, Water, Internet, etc.
Group	GroupID	INT	Primary Key
	GroupName	INT	Name of the group/household
Member	GroupID	INT	FK → Group.GroupID
	UserID	INT	FK → User.UserID
	Role	VARCHAR	"member", "admin"
BankAccount	AccountID	INT	Primary Key
	UserID	INT	FK → User.UserID
	BankName	VARCHAR	Name of the bank
	AccountNumber	VARCHAR	Masked or tokenized account number
Transaction	TransactionID	INT	Primary Key

	AccountID	INT	FK → Bank_Account.AccountID
	Amount	DECIMAL	Transaction amount
	Date	DATE	Transaction date
	Description	TEXT	Bank-provided text
	Linked_Receipt_ID	INT	FK → Receipt.ReceiptID (nullable)
Report	ReportID	VARCHAR	Primary Key
	UserID	INT	FK → User.UserID
	ReportType	VARCHAR	"PDF", "CSV", "PNG"
	PasswordProtected	BOOLEAN	Whether the report is password-locked
	CreatedDate	DATE	Time to generate report
Budget	BudgetID	INT	Primary Key
	UserID	INT	FK → User.UserID
	LimitAmount	DECIMAL	The max budget set for this category
	StartDate	DATE	Start date of the budget period (e.g., start of the month)
	EndDate	DATE	End date of the budget period (e.g., end of the month)
Item	ItemID	INT	Primary key
	ReceiptID	INT	FK → Receipt.ReceiptID
	ItemName	TEXT	Name or description of the item (e.g., "Electricity Unit", "Banana")
	Quantity	DECIMAL	Quantity of the item (e.g., 2, 3.5 units)
	UnitPrice	DECIMAL	Price per unit
	CategoryID	INT	Foreign Key → Category.CategoryID
Suggestion	SuggestionID	INT	Primary key
	UserID	INT	FK → User.UserID
	Description	TEXT	Suggestion text (e.g., "Switch to XYZ provider to save 20%")
	Savings_Amount	DECIMAL	Estimated savings if the suggestion is followed
Notification	NotificationID	INT	Primary key
	UserID	INT	FK → User.UserID

	Message	VARCHAR	The content of the notification (e.g., new suggestion).
	Notification_type	VARCHAR	Type of notification (e.g., "Reminder", "Suggestion", "Alert").
	Is_Read	BOOLEAN	Flag to indicate whether the notification has been read.

2.5 Design and Implementation Constraints

Several constraints will influence the development of the receipt scanner and expense analyzer app. These constraints include hardware limitations, software tools, technologies, and design standards that must be adhered to during the development process.

Hardware Limitations:

- The app will need to perform receipt scanning using the camera on mobile devices. As such, it must be optimized for performance to handle image processing without excessive lag or memory usage. It should also be compatible with both Android and iOS devices, with support for varying camera resolutions and performance capabilities.

Performance Requirements:

- Given that users may input a high volume of receipts and transaction data, the app must be able to handle large datasets efficiently. Memory management and optimization for quick access to stored receipts and reports are essential. Real-time notifications for group use will also require efficient background operations.

Technology Stack:

- The app will be developed using native mobile technologies, such as Swift for iOS and Kotlin for Android, to ensure high performance and a responsive user interface. The cloud storage service for synchronization will be built using Firebase or AWS, providing secure, reliable storage and real-time synchronization.
- The COMET method will be used for the software design, ensuring that we follow an iterative, modular approach to development. This method will help us manage complexity and reduce the risk of errors during development.
- UML (Unified Modeling Language) will be utilized for designing the system architecture, including class diagrams, use case diagrams, and sequence diagrams, to ensure clarity and consistency in the system's design.

Interfaces to Other Applications:

- The app will need to integrate with cloud storage platforms for syncing data across devices. Additionally, users may wish to export reports to other tools (e.g., Excel or accounting software). Therefore, it will be essential to implement export functionality in formats like CSV or PDF.

Security Considerations:

- As the app deals with sensitive financial data, robust security measures will be implemented. This includes end-to-end encryption for receipt data, secure authentication methods (such as OAuth2 for group accounts), and compliance with data privacy regulations (such as GDPR or CCPA).
- Regular security updates and patches will be needed, and all data stored in the cloud will be encrypted at rest and during transmission.

Communication Protocols:

- Communication between the app and the cloud services (for data syncing, receipt upload, and report generation) will use HTTPS to ensure secure transmission. Additionally, real-time communication for notifications will rely on WebSocket or Firebase Cloud Messaging (FCM).

Design Conventions & Programming Standards:

- The development team must follow standard mobile app development practices, including using MVC (Model-View-Controller) for app architecture and MVVM (Model-View-ViewModel) for managing data and UI interactions.
- Code readability and documentation standards will be emphasized to ensure maintainability, especially since the app will require future updates and bug fixes.
- The app must comply with accessibility standards, ensuring that it is usable by people with disabilities. This includes implementing support for screen readers and providing high contrast visuals.

By adhering to these constraints and standards, the development of the receipt scanner and expense analyzer app will be streamlined, secure, and capable of delivering a high-quality user experience across different platforms.

2.6 Assumptions and Dependencies

Assumptions:

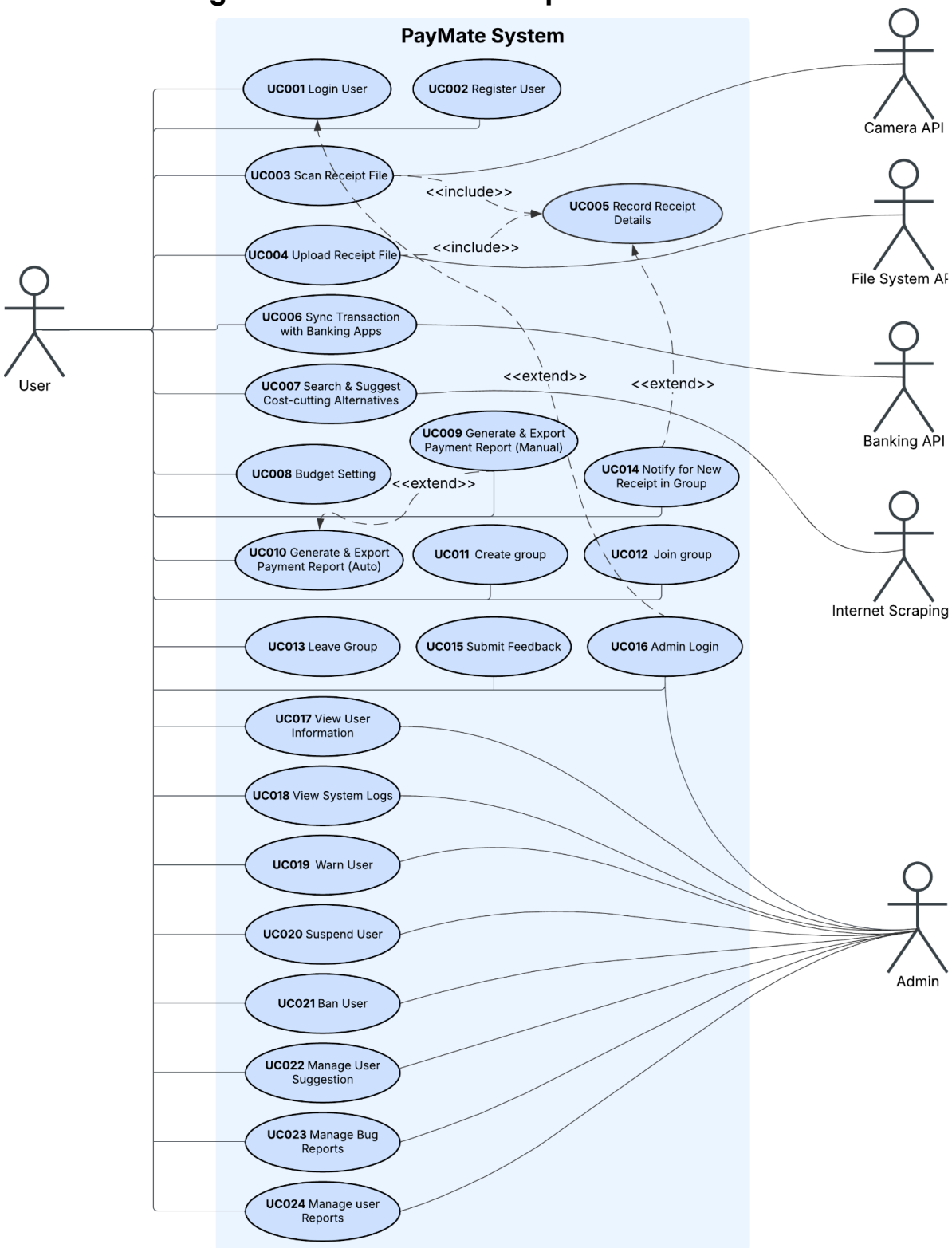
1. **Users Have Access to the Feedback System** – It is assumed that users can navigate to the feedback submission section through the mobile application without restrictions.
2. **Internet Connectivity is Stable** – The feedback process requires an active internet connection for successful submission. If the connection is lost, users may need to retry.
3. **Feedback Processing is Automated** – The system automatically stores and processes the feedback without requiring manual intervention unless flagged for moderation.
4. **User Authentication is Functional** – If the feedback system requires users to log in, it is assumed that authentication services are operational.
5. **Data Storage and Retrieval are Reliable** – The system correctly stores user feedback in the database and allows administrators to access and review it when needed.

Dependencies:

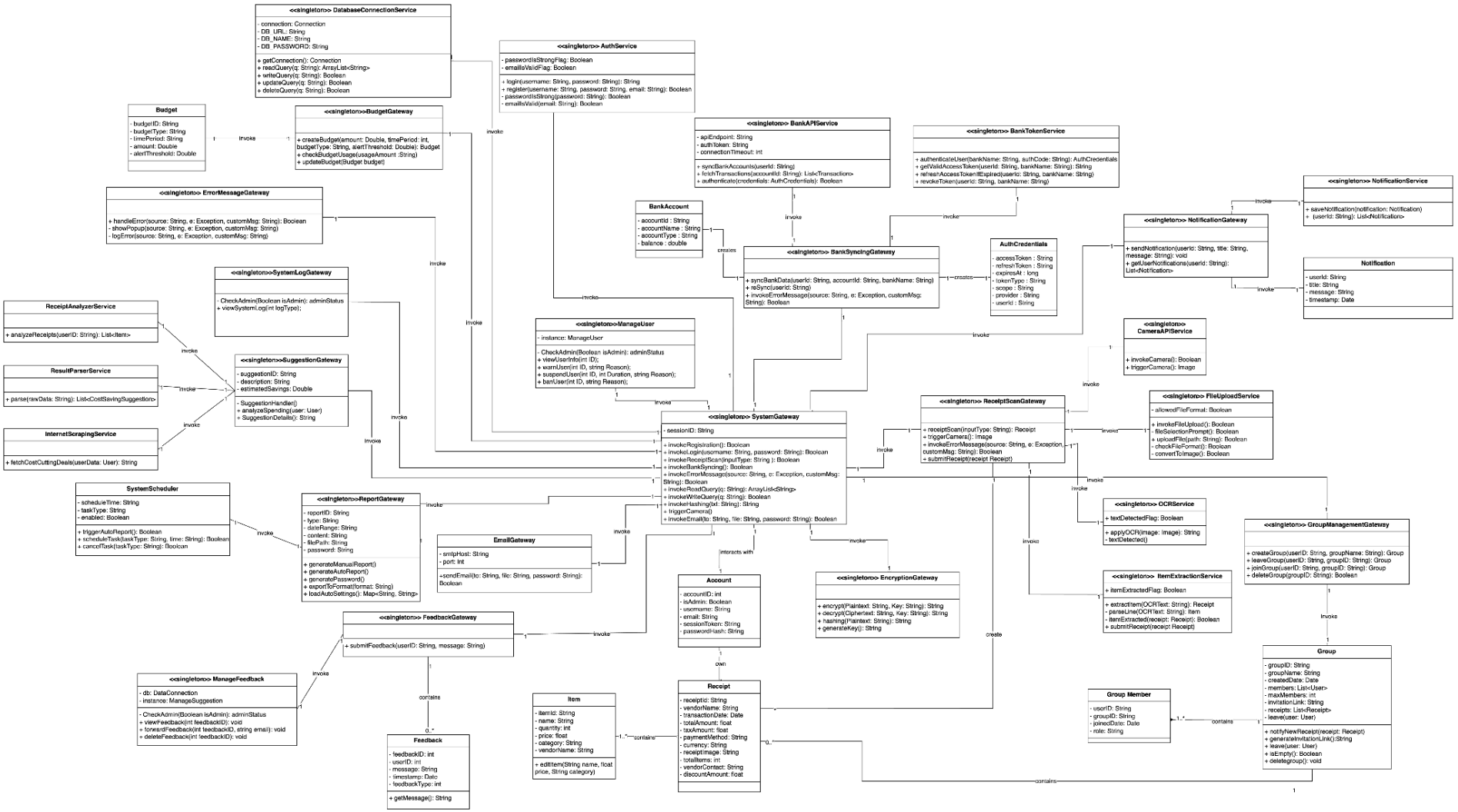
1. **Database System** – The feedback system depends on a functional database to store and retrieve feedback data.
2. **Notification Service** – If users receive confirmation messages (email), the system depends on external notification services.
3. **Content Moderation Service** – If automated moderation is required, the system may depend on third-party moderation tools or AI-based filtering.
4. **Third-Party API Integrations** – If feedback analytics or sentiment analysis is involved, the system may rely on external APIs for processing.
5. **Hosting and Server Availability** – The feedback system depends on a web server or cloud service for operation. Downtime could impact submission and processing.

3 Detail Design Specification

3.1 Use Case Diagram for Functional Requirements



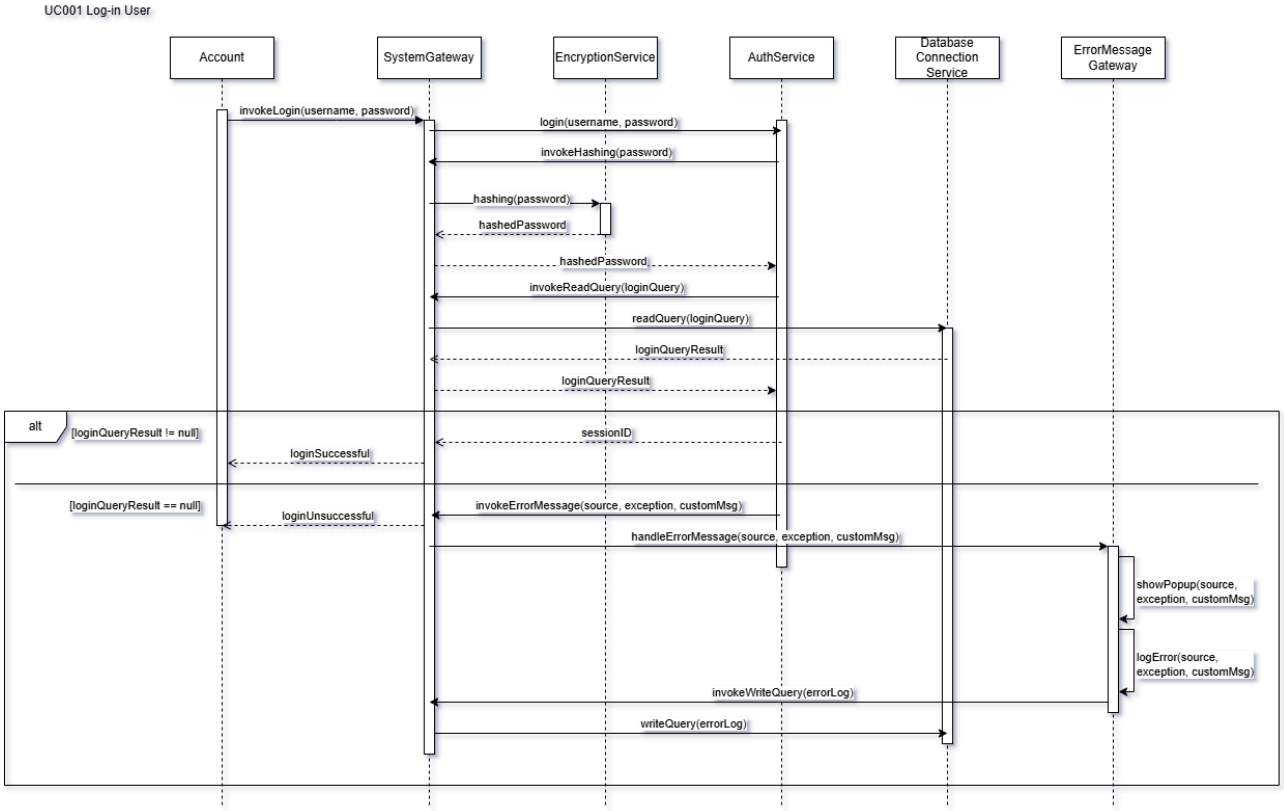
3.2 Class Diagram Design (OOP design)



https://drive.google.com/file/d/1M4geCkKKKWDgEGvqoMvJvwb0U1Rz4iGa/view?usp=sharing

3.3 Sequence Diagram Design

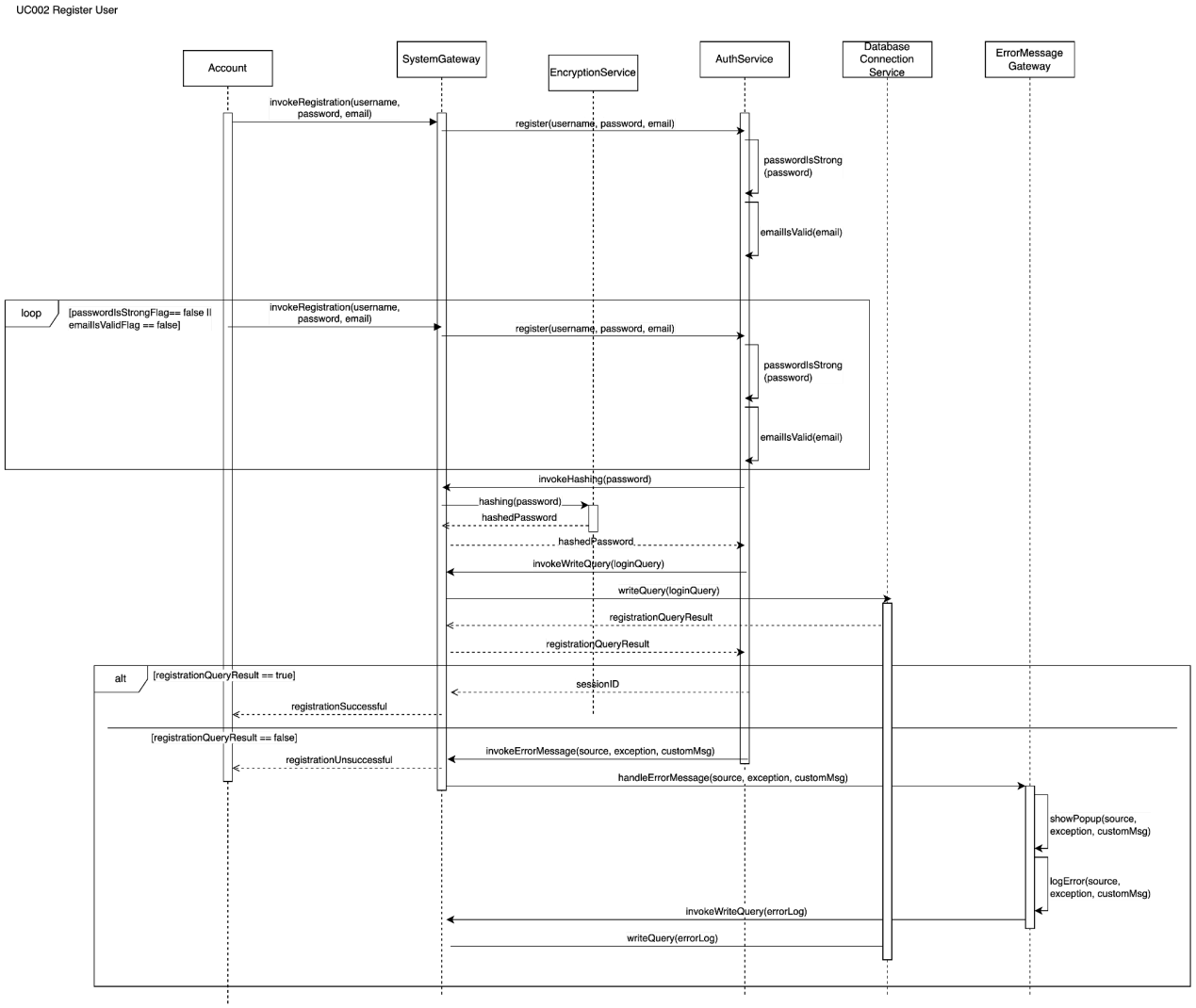
3.3.1 UC001 Log-in User Sequence Diagram



Author: Nattapol Phanmadee

This diagram depicts the sequence flow of UC001 Log-in User use case. The user interacts with the log-in system via system gateway, providing the username & password. The system gateway handles the task by delegating to the authentication service, encryption service class, and database connection service, which interacts with the database to find a user having matching credentials. If the user with the given credentials does not exist, the sequence switches to the alternative flow to handle error displaying & error logging.

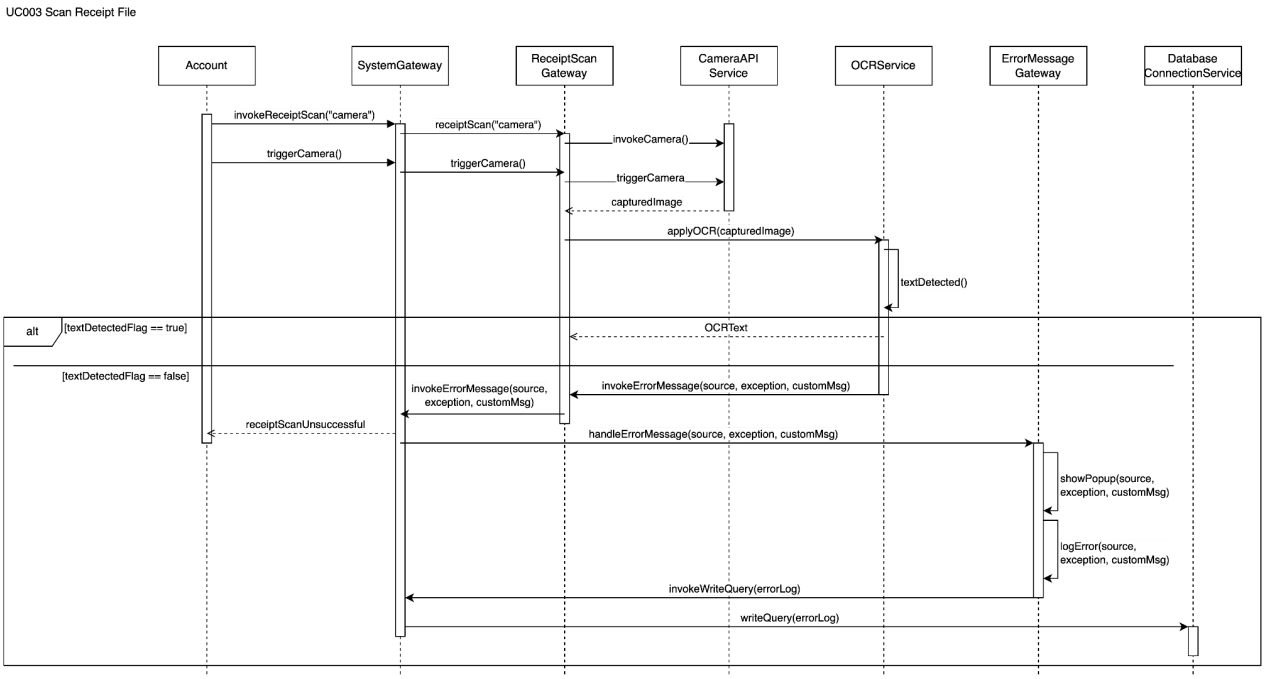
3.3.2 UC002 Register User Sequence Diagram



Author: Nattapol Phanmadee

This diagram depicts the sequence flow of UC002 Register User use case. The user interacts with the registration system via the SystemGateway, providing a username, password, and email. The SystemGateway delegates the input to the EncryptionService and AuthService for validation and password hashing. Once validated, the system writes the new user data to the database through the Database Connection Service. If the registration is successful, a session ID is generated and returned to the user. If registration fails (e.g., due to invalid input or a database error), the sequence follows the alternative flow to handle error messaging and logging via the ErrorMessage Gateway.

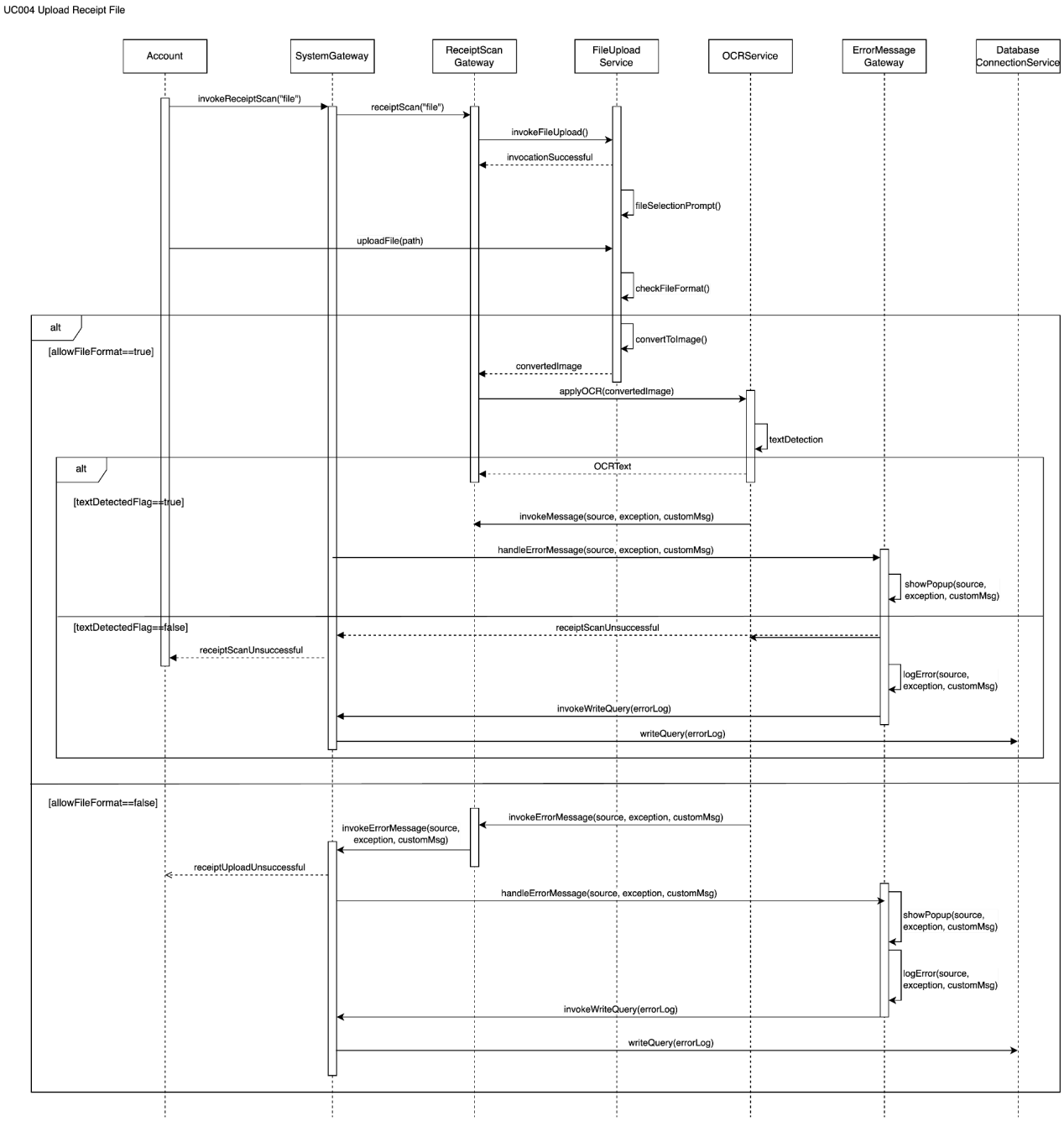
3.3.3 UC003 Scan Receipt File Sequence Diagram



Author: Nattapol Phanmadee

This diagram illustrates the UC003 Scan Receipt File use case. When a logged-in user selects the "Input Receipt" button and presses "Capture", the SystemGateway triggers the camera via the CameraAPI Service. The captured image is passed to the OCRService, which attempts to extract text from it. If text is detected, it is temporarily stored in the system. If no text is found, the flow enters an exception branch handled by the ErrorMessage Gateway, which displays a prompt and logs the error. The user is then prompted to re-capture the receipt image.

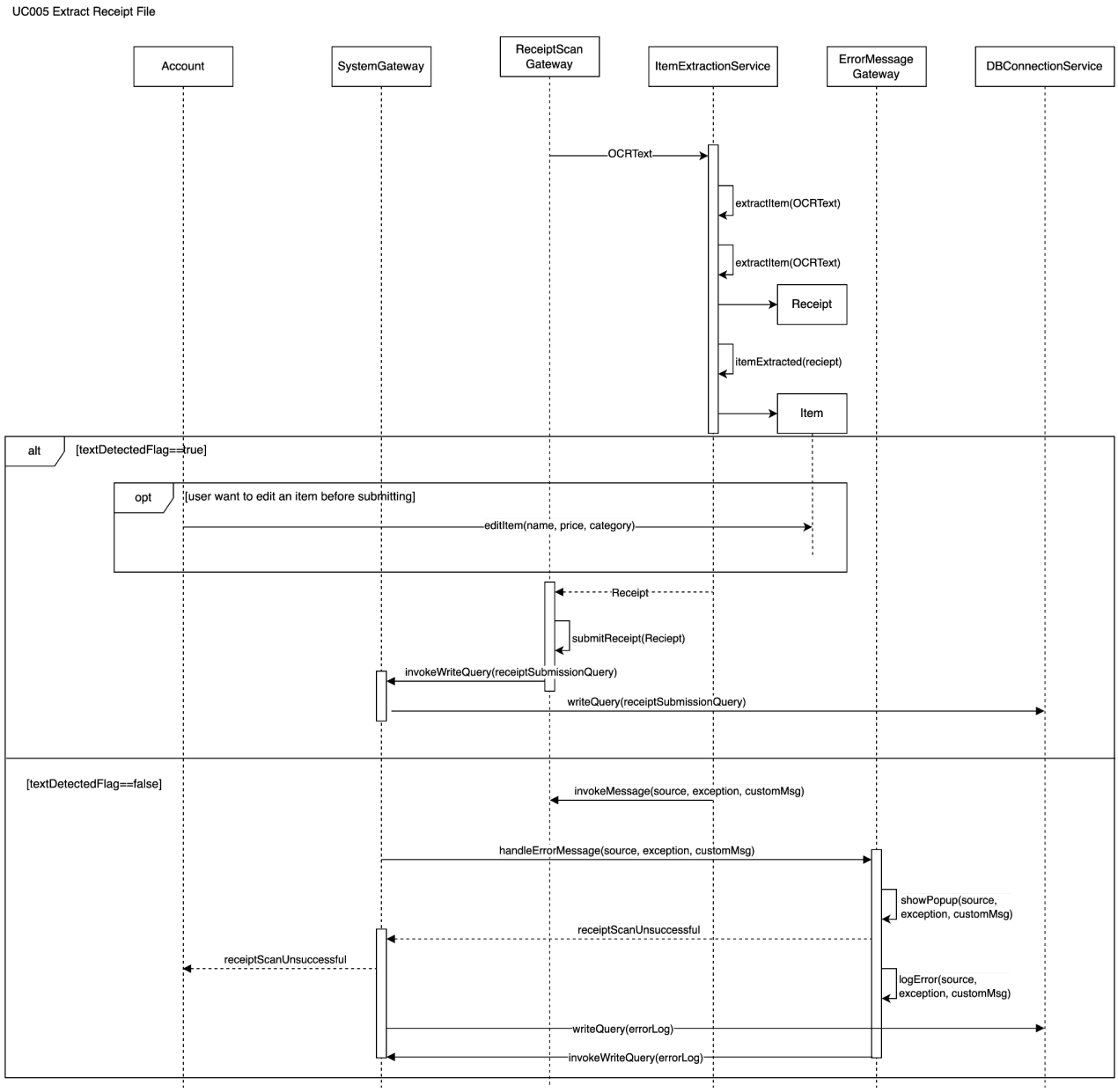
3.3.4 UC004 Upload Receipt File Sequence Diagram



Author: Nattapol Phanmadee

This diagram illustrates the UC004 Upload Receipt File use case. After selecting "Upload," the user is directed to the file system interface. Upon selecting a file, the SystemGateway forwards it through the ReceiptScan Gateway to the CameraAPI Service, which then passes it to the OCRService. If the file is a valid type and OCR text is detected, the scanned content is stored temporarily. If the file is invalid , or no text is detected, the ErrorMessage Gateway manages the error by displaying a prompt and logging the issue. The user is then prompted to upload a valid file or retry the operation.

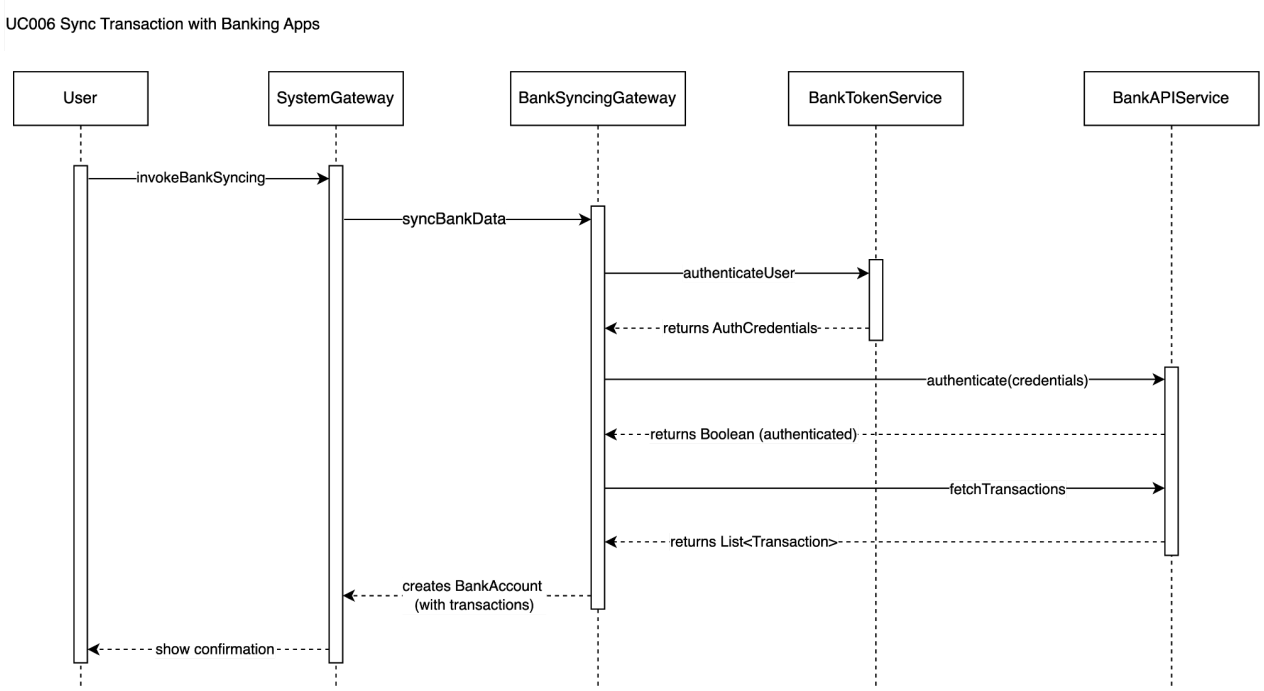
3.3.5 UC005 Extract Receipt Details Sequence Diagram



Author: Nattapol Phanmadee

This diagram depicts the sequence flow of UC005 Extract Receipt Details use case. The user interacts with the receipt scanning system via the system gateway, which forwards OCR-scanned text to the ReceiptScan Gateway. The ReceiptScan Gateway handles the task by delegating to the item extraction service, which analyzes the text to extract receipt items and details. Each extracted item is categorized before being returned. If the extraction succeeds, the system displays the list of items to the user, who may optionally edit the item name, price, or category. The finalized receipt is then submitted to the database connection service for storage.

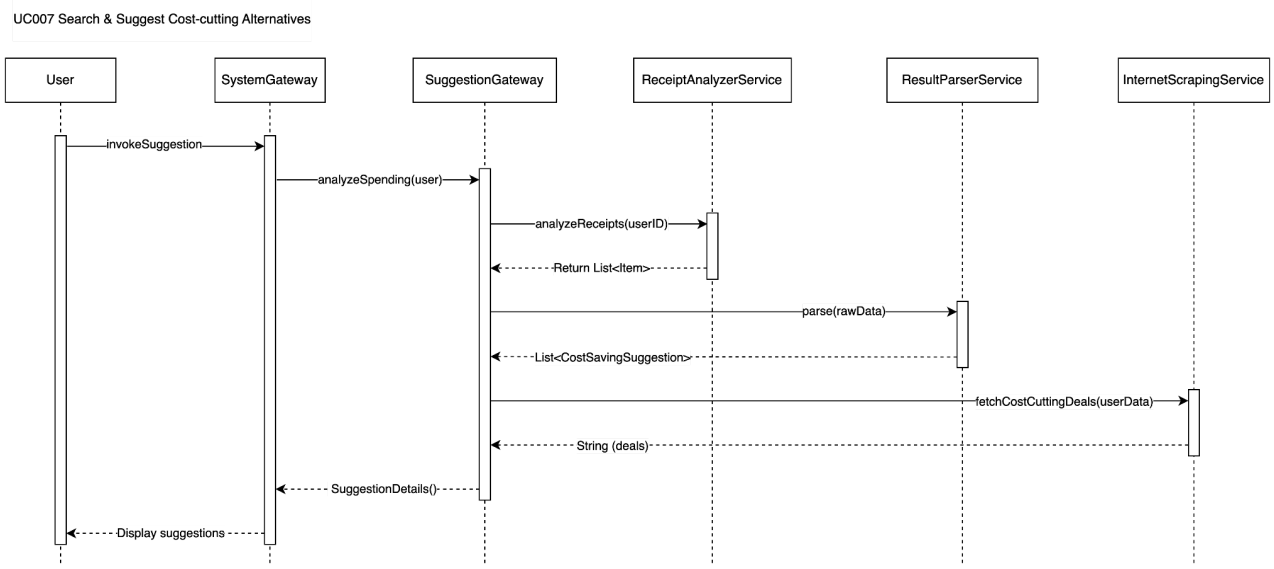
3.3.6 UC006 Sync Transaction with Banking Apps Sequence Diagram



Author: Aekkarin Yimyaem

This diagram depicts the sequence flow of UC006 Sync Transaction with Banking Apps use case. The user initiates the transaction sync via the system gateway by clicking the banking transaction sync button. The system gateway delegates the task to the bank syncing gateway, which interacts with the bank token service to authenticate the user. The bank token service generates authentication credentials and forwards them to the bank API service for verification. Upon successful authentication, the bank API service fetches the transaction data and returns it through the gateway. The system then creates a bank account with the received transactions and shows a confirmation to the user.

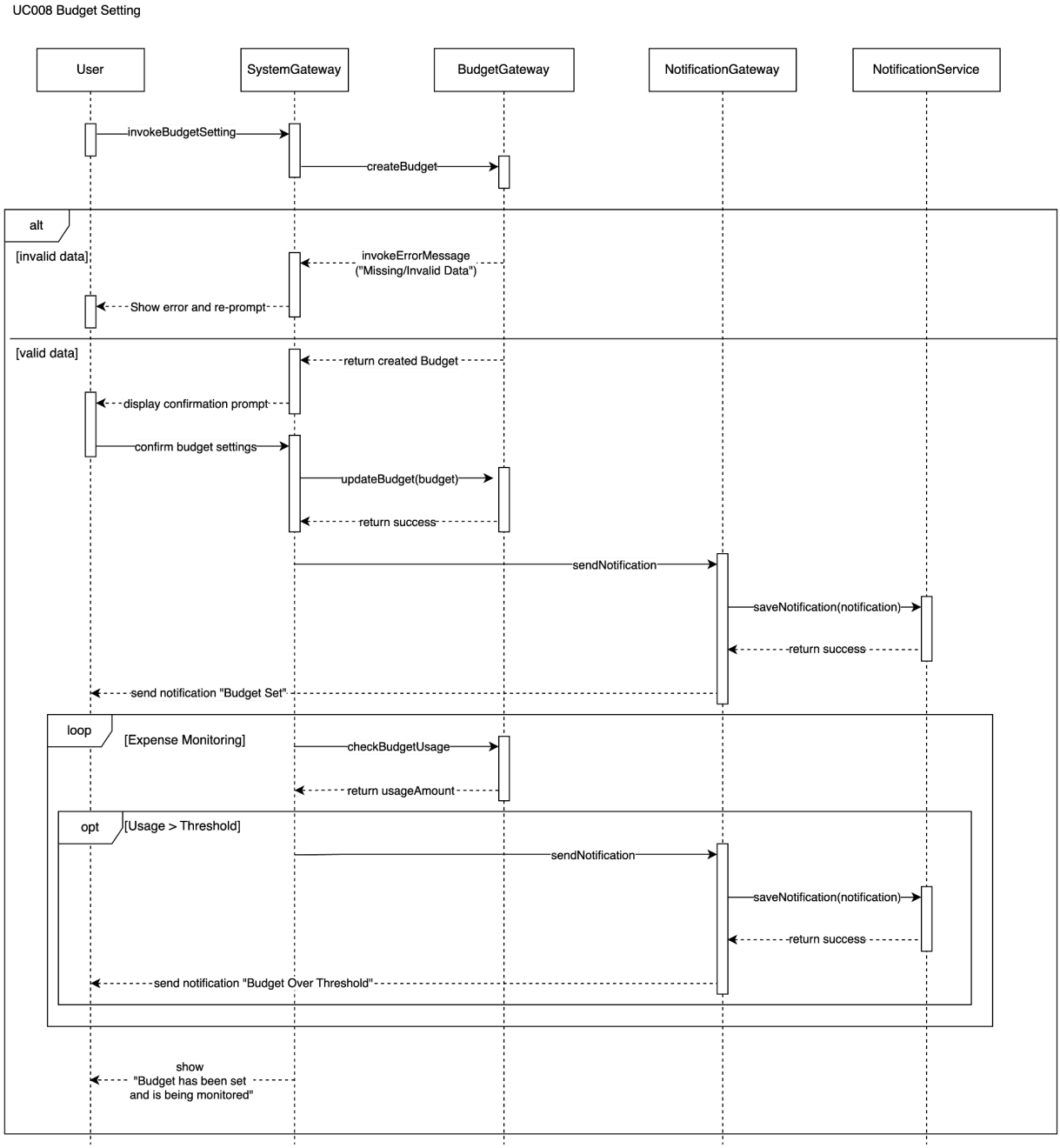
3.3.7 UC007 Search & Suggest Cost-cutting Alternatives Sequence Diagram



Author: Aekkarin Yimyaem

This diagram depicts the sequence flow of UC007 Search & Suggest Cost-cutting Alternatives use case. The user initiates the cost-cutting suggestion process by clicking the cost-cutting button through the system gateway. The system gateway delegates the request to the suggestion gateway, which analyzes the user's spending history using the receipt analyzer service. The receipt analyzer retrieves past spending data and identifies items, which are then passed to the result parser service. The result parser interacts with the internet scraping service to fetch potential cost-cutting deals based on the user's data. Once the deals are retrieved and parsed, the suggestion gateway compiles cost-saving suggestions and sends them back to the system gateway. Finally, the system displays the personalized suggestions to the user.

3.3.8 UC008 Budget Setting Sequence Diagram

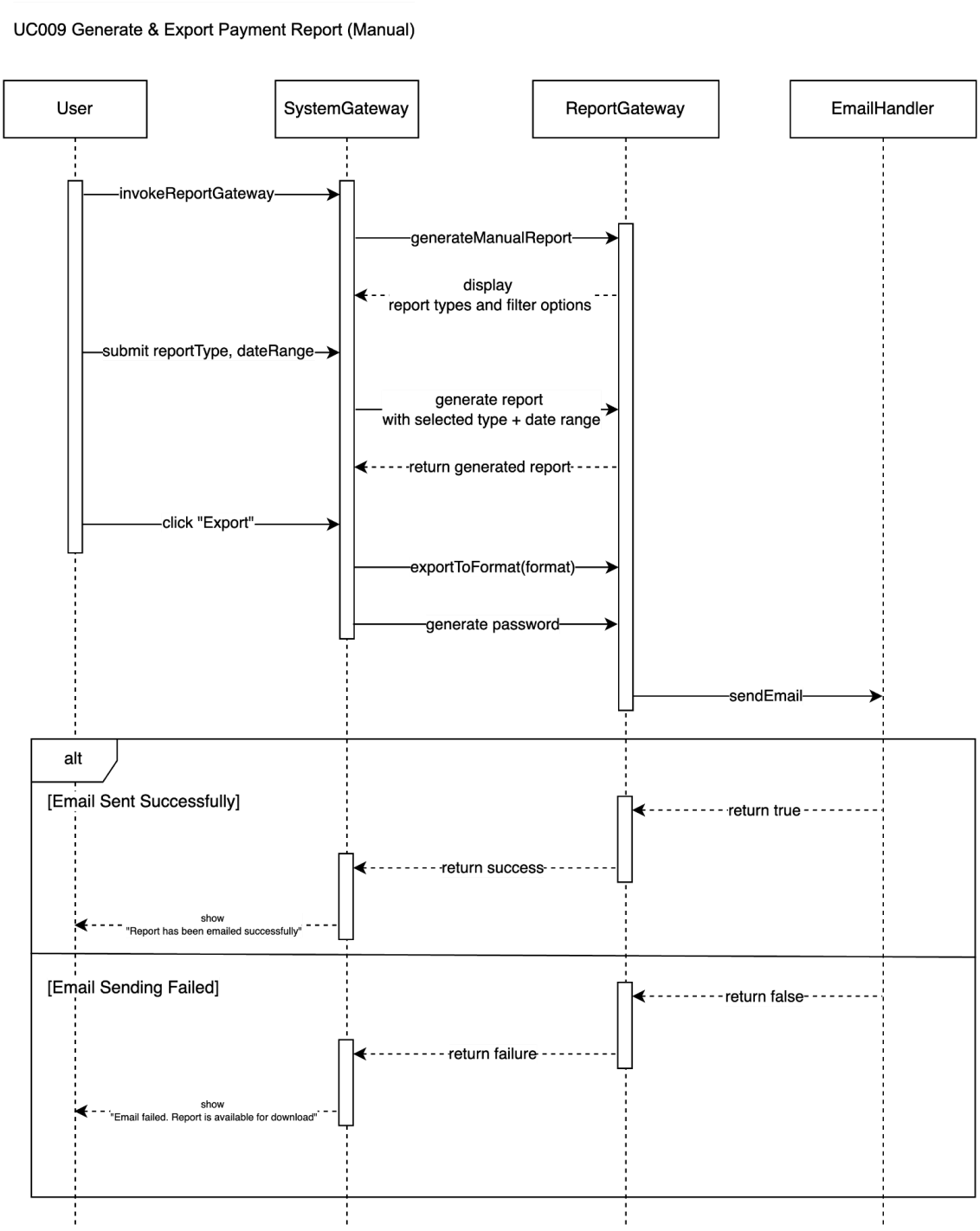


Author: Aekkarin Yimyaem

This diagram depicts the sequence flow of UC008 Budget Setting use case. The user initiates the process by selecting the budget setting option from the dashboard through the system gateway. The system gateway forwards the request to the budget gateway to create a budget(User enters data). If required data is missing or invalid, the system triggers an error message, prompting the user to correct the input. Once valid data is provided, the system gateway displays a confirmation

prompt. Upon user confirmation, the budget settings are updated and saved via the budget gateway. The system then sends a "Budget Set" notification through the notification gateway, which is persisted by the notification service. Subsequently, the system continuously monitors budget usage. If the recorded usage surpasses the alert threshold, another notification is triggered and saved, alerting the user that their budget has been exceeded.

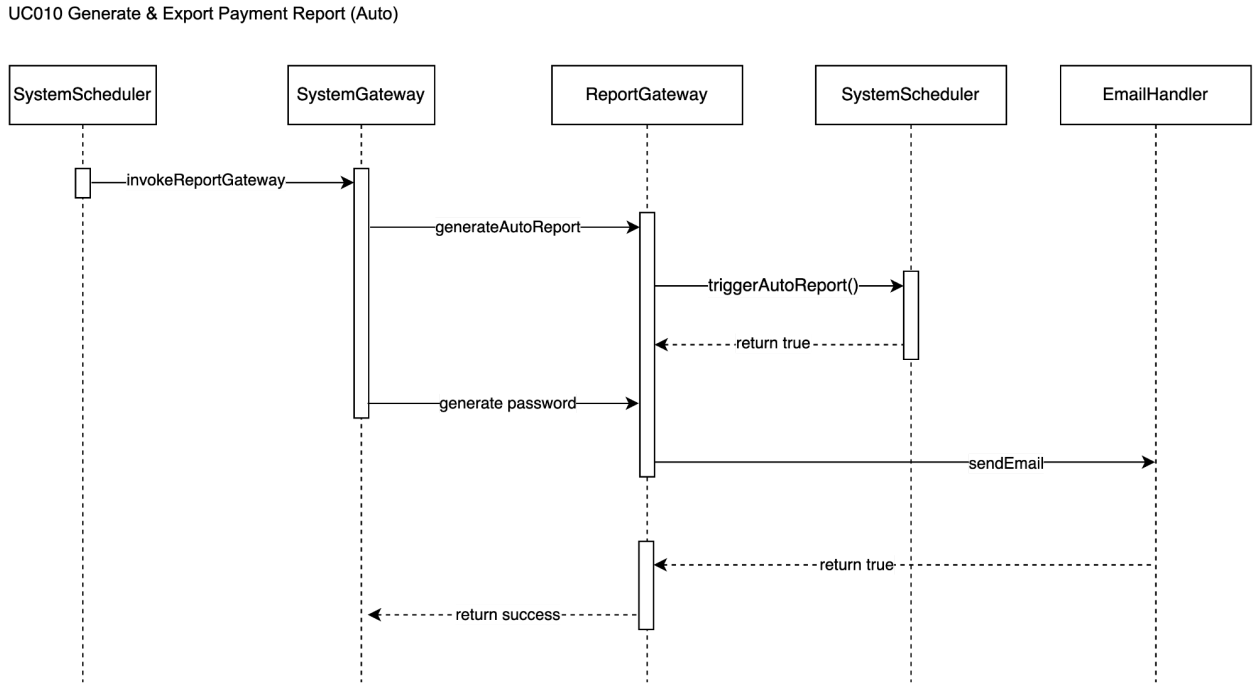
3.3.9 UC009 Generate & Export Payment Report (Manual) Sequence Diagram



Author: Aekkarin Yimyaem

This diagram depicts the sequence flow of UC009 Generate & Export Payment Report (Manual) use case. The user interacts with the system through the system gateway by selecting the "Export Reports" option and submitting the desired report type and date range. The system gateway delegates the task to the report gateway, which generates the report based on the provided criteria. Upon clicking "Export," the system gateway requests the report gateway to export the report into the specified format and generate a password for file security. The email handler service is then called to send the report via email. If the email is sent successfully, the system displays a success message. If the email fails to send, the sequence switches to the first step is for the user to enter information.

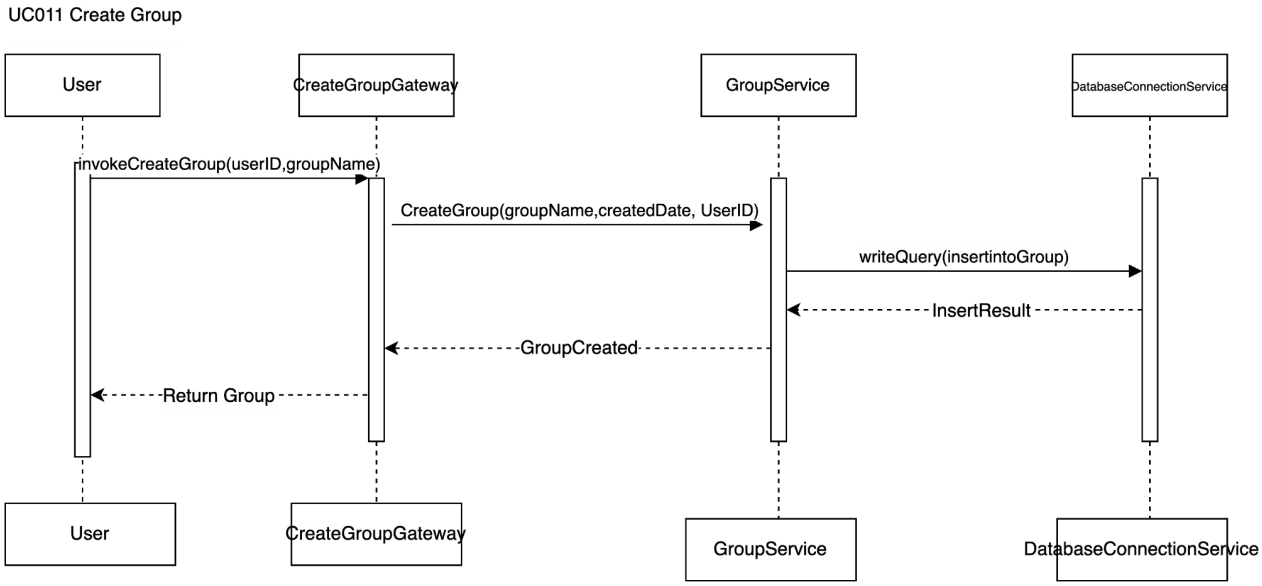
3.3.10 UC010 Generate & Export Payment Report (Auto) Sequence Diagram



Author: Aekkarin Yimyaem

This diagram depicts the sequence flow of UC010 Generate & Export Payment Report (Auto) use case. The system scheduler automatically invokes the system gateway to initiate report generation based on predefined settings. The system gateway then delegates the request to the report gateway, which generates the report according to preset criteria. The system subsequently generates a password to secure the exported file. The email handler service is invoked to send the report via email.

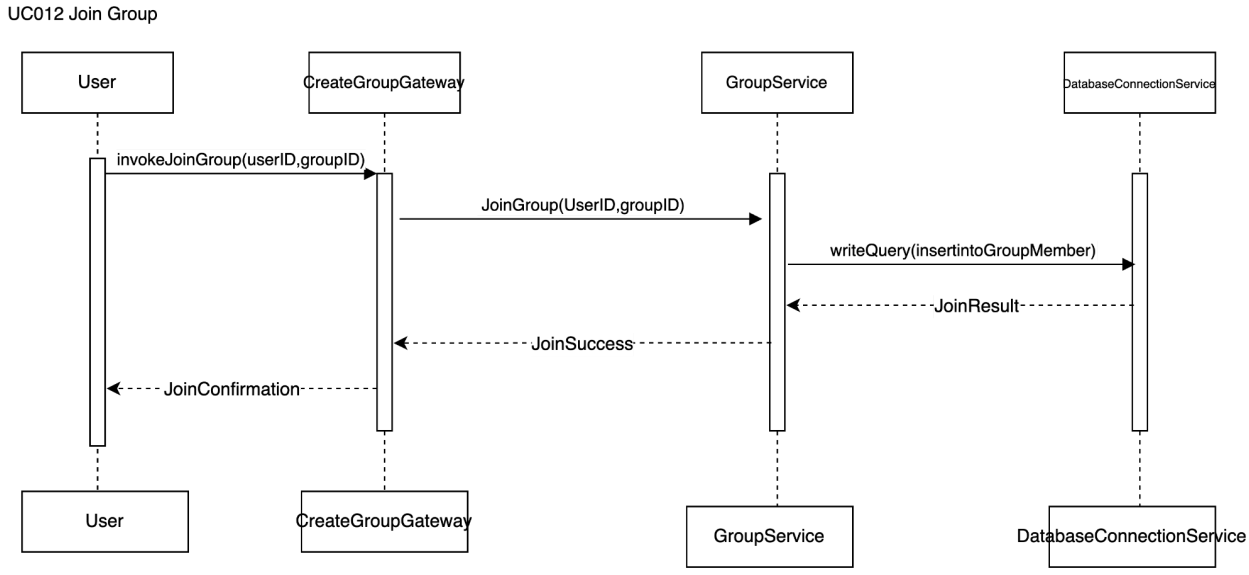
3.3.11 UC011 Create Group Sequence Diagram



Author: Kitssada Pavasutthipand

This diagram illustrates the sequence flow of the UC011 Create Group use case. The user sends a request to create a group via the `CreateGroupGateway`, providing `userID` and `groupName`. The gateway forwards the request to `GroupService`, which constructs the data and sends an `insertGroup` query to `DatabaseConnectionService`.

3.3.12 UC012 Join Group Sequence Diagram

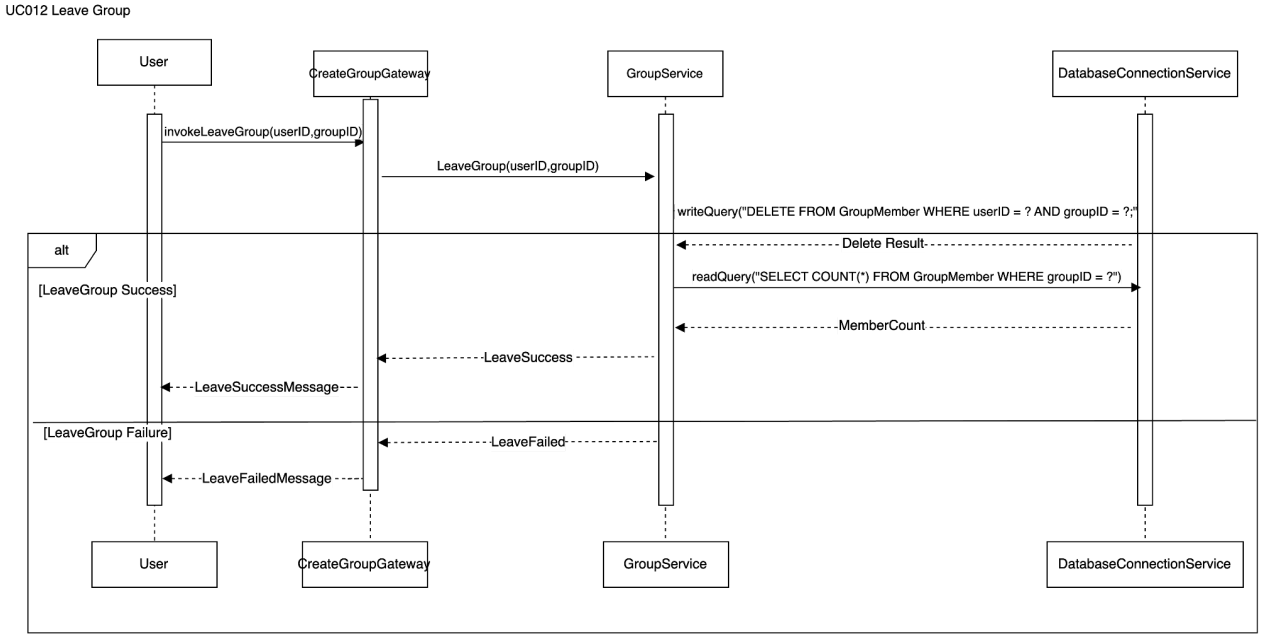


Author: Kitssada Pavasutthipand

This diagram shows the flow for the UC012 Join Group use case. The user sends a join request through the CreateGroupGateway with their user ID and group ID. The request is passed to the GroupService, which sends a database query to add the user to the group.

If successful, the result is returned and the user is notified of the successful join. In case of an error, the system would handle it and inform the user, though this is not shown in the diagram

3.3.13 UC013 Leave Group Sequence Diagram

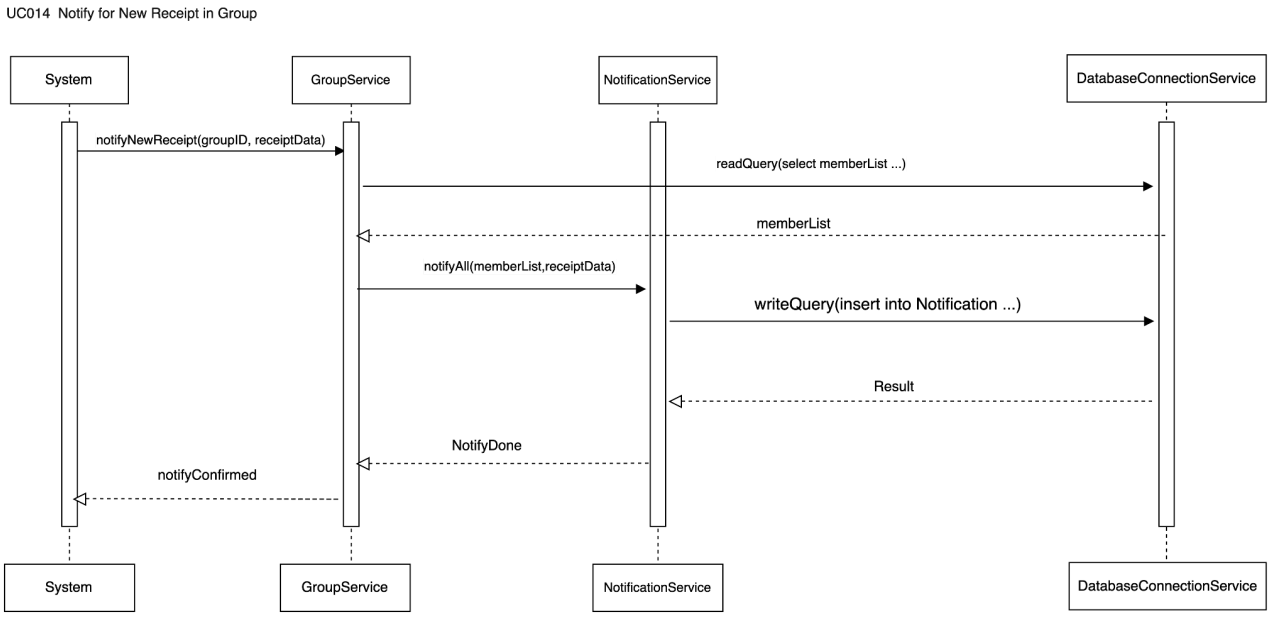


Author: Kitssada Pavasutthipand

This diagram shows the flow of the **UC013 Leave Group** use case. The user sends a leave group request through the CreateGroupGateway with their user ID and group ID. The gateway forwards the request to the GroupService, which first sends a delete query to remove the user from the group.

After deletion, the GroupService checks the number of remaining members in the group by querying the database. If the operation is successful, a success message is returned to the user. If it fails, an error message is sent instead. The alternate flow for success or failure is shown using a conditional branch.

3.3.14 UC014 Notify for New Receipt in Group Sequence Diagram

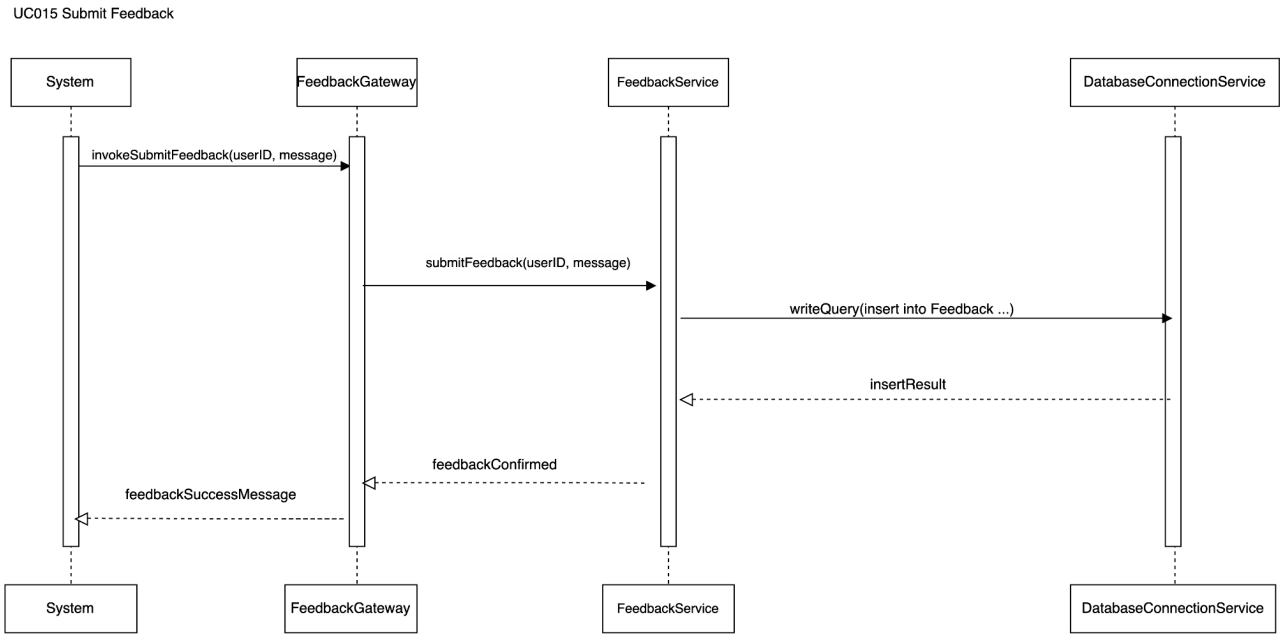


Author: Kitssada Pavasutthipand

This diagram shows the flow of the UC014 Notify New Receipt in Group use case. The system sends a request to the GroupService to notify a new receipt using the group ID and receipt data. The GroupService forwards this to the NotificationService, which first queries the database to get the list of group members.

Once the member list is retrieved, the NotificationService loops through the list and inserts notification entries into the database. After the operation is completed, a success result is returned to the GroupService, which then confirms the notification back to the system.

3.3.15 UC015 Submit Feedback Sequence Diagram

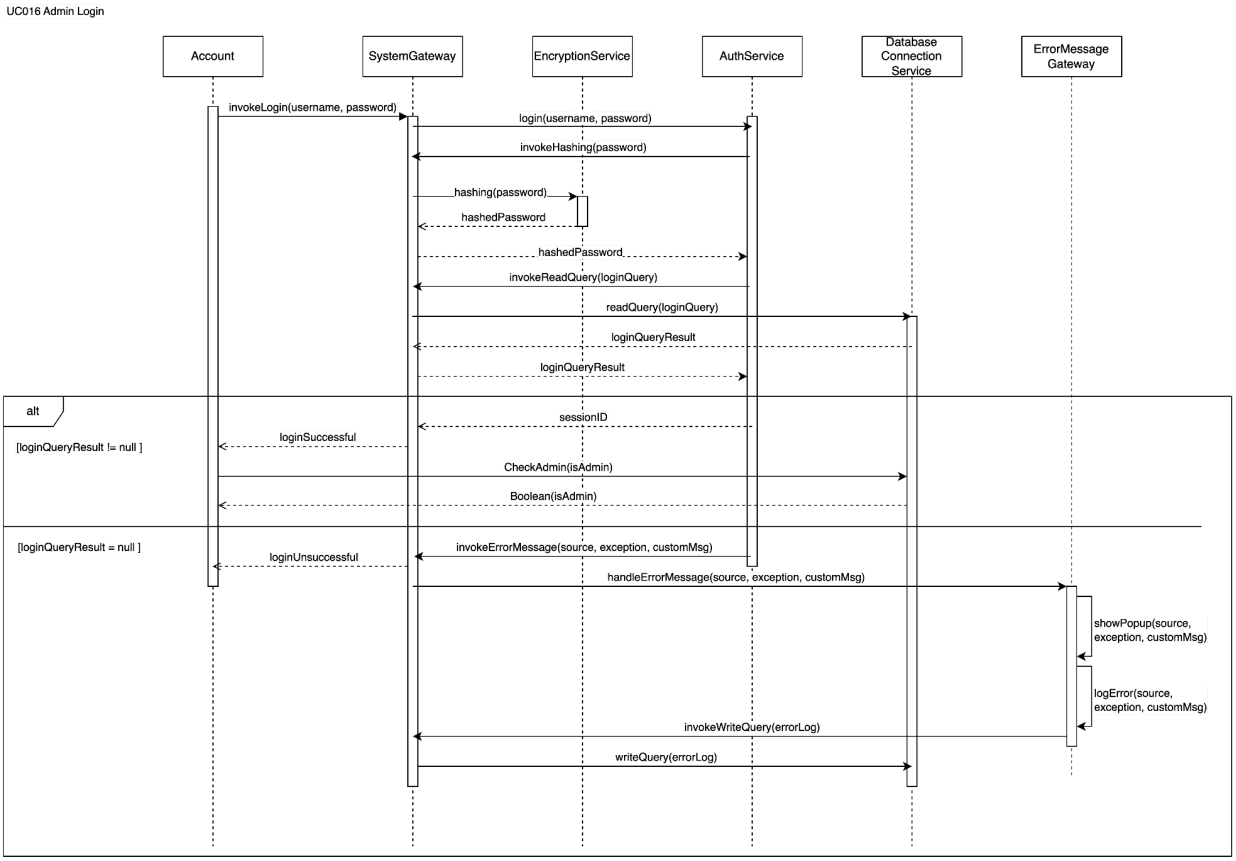


Author: Kitssada Pavasutthipand

This diagram shows the flow of the UC015 Submit Feedback use case. The system sends a feedback submission request through the FeedbackGateway with the user ID and message. The gateway forwards it to the FeedbackService, which sends a database insert query to store the feedback.

Once the database operation succeeds, the result is returned, and a confirmation message is passed back through the gateway to the system, indicating successful submission.

3.3.16 UC016 Admin Login Sequence Diagram

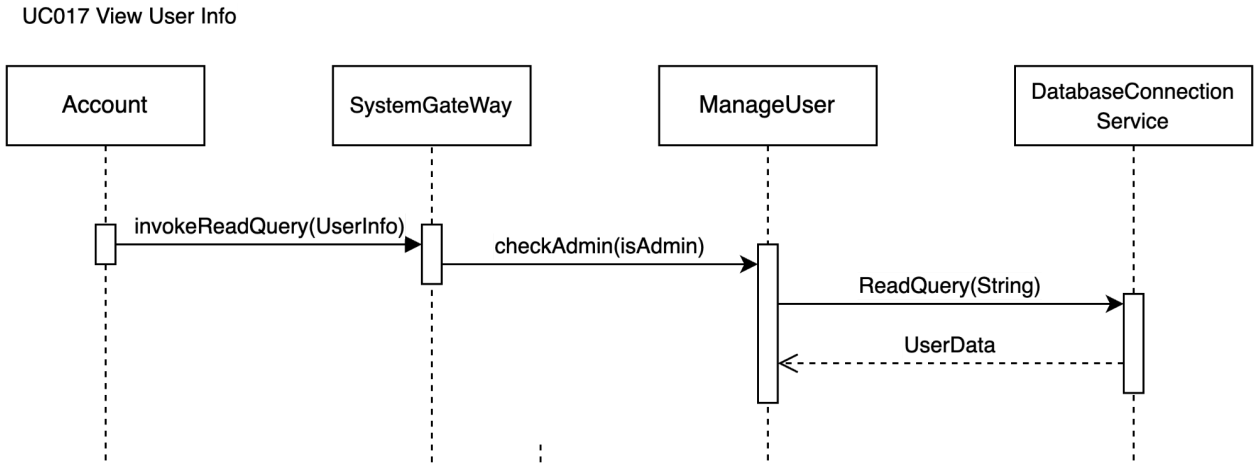


Author: Chayathon Chaiyasak

This diagram shows the flow of the UC016 Admin Login use case. The user enters a username and password to log in. The system sends this information through the SystemGateway, which hashes the password using the EncryptionService.

Then, the system checks the login credentials with the database through the AuthService. If the login is successful, the system checks if the user is an admin and creates a session. If the login fails, the system shows an error message using the ErrorMessageGateway and logs the error.

3.3.17 UC017 View User Information Sequence Diagram

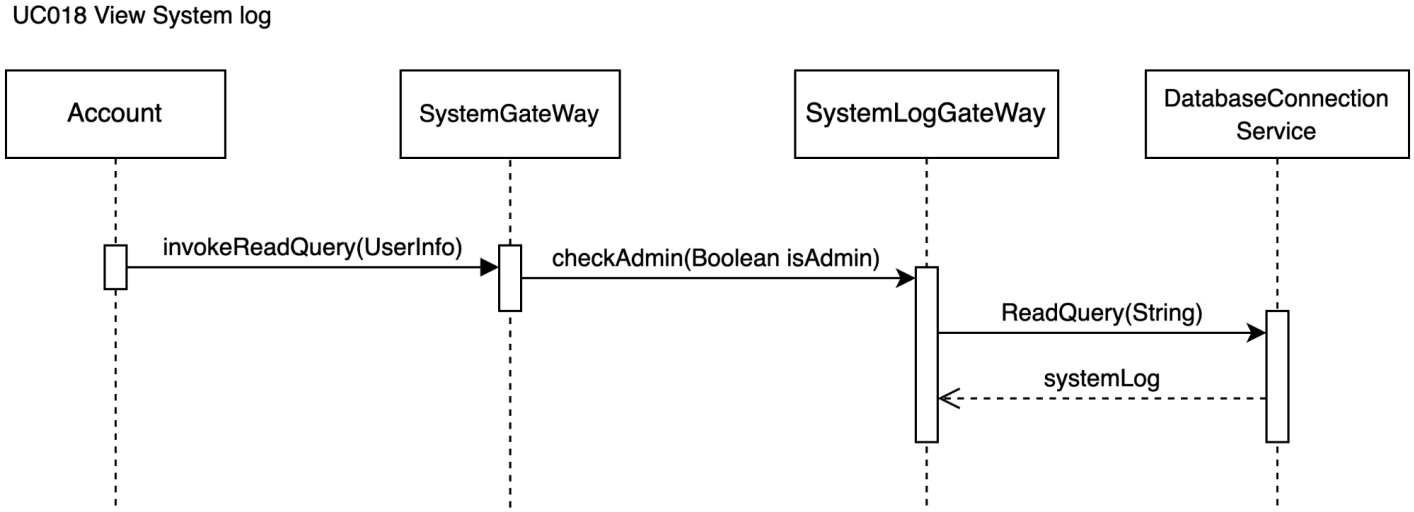


Author: Chayathon Chaiyasak

This diagram shows the flow of the UC017 View user information use case. The admin requests to view user information. The request is sent through the SystemGateway, which first checks if the user is an admin.

If the user has admin rights, the system forwards the request to the ManageUser service, which queries the database through the DatabaseConnectionService. Once the data is retrieved, the user information is sent back to the admin.

3.3.18 UC018 Admin Login Sequence Diagram

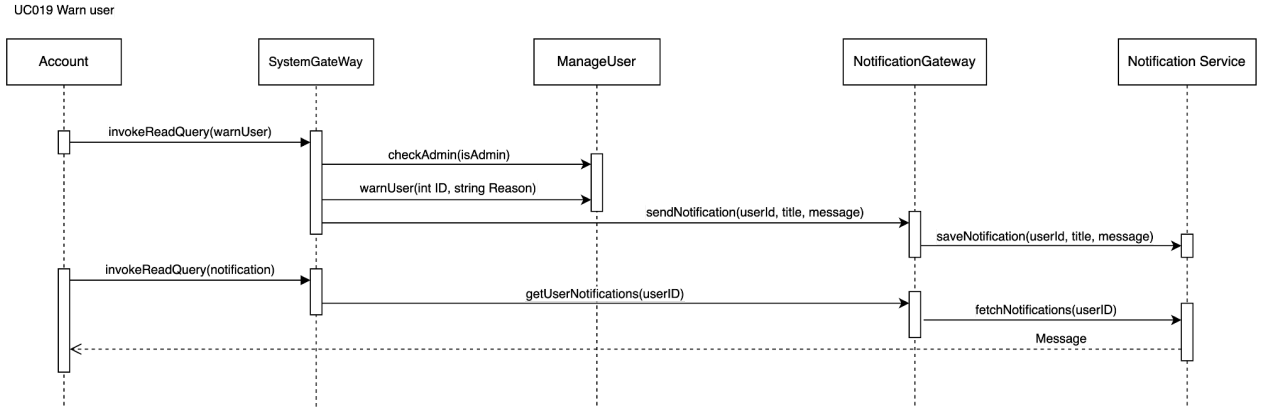


Author: Chayathon Chaiyasak

This diagram shows the flow of the UC018 View System log use case.The admin requests to view the system log. The request is sent through the SystemGateway, which checks if the user has admin rights.

If the user is an admin, the system forwards the request to the SystemLogGateWay, which queries the log data from the database via the DatabaseConnectionService. Once retrieved, the system log is sent back to the admin.

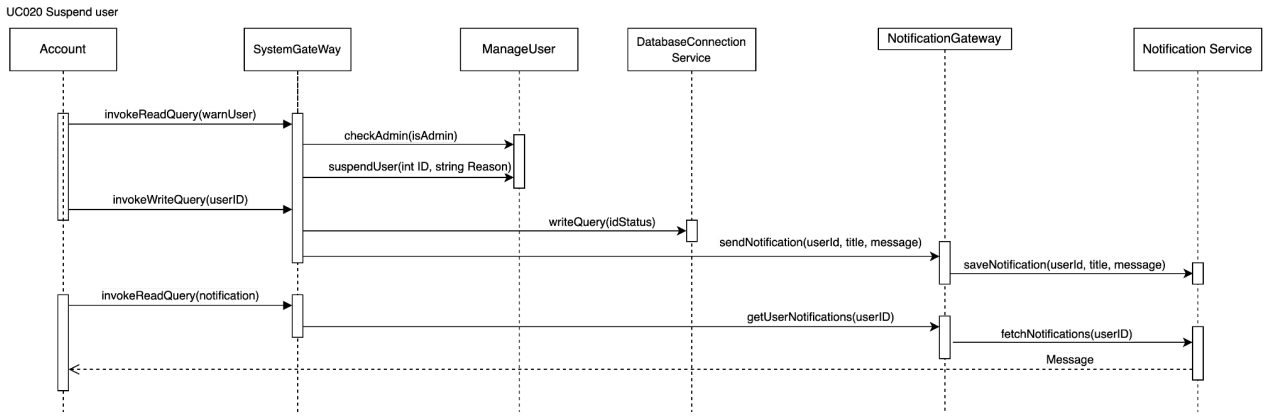
3.3.19 UC019 Warn User Sequence Diagram



Author: Chayathon Chaiyasak

This diagram shows the flow of the UC019 Warn user use case. The admin chooses to warn a user. The system first verifies admin rights via the SystemGateway. If the user is an admin, the system sends the warning (with a reason) to ManageUser, which passes it to the Notification Gateway. The NotificationGateway saves the warning message to the database using the NotificationService. After that, the system fetches and displays the updated notifications for the user.

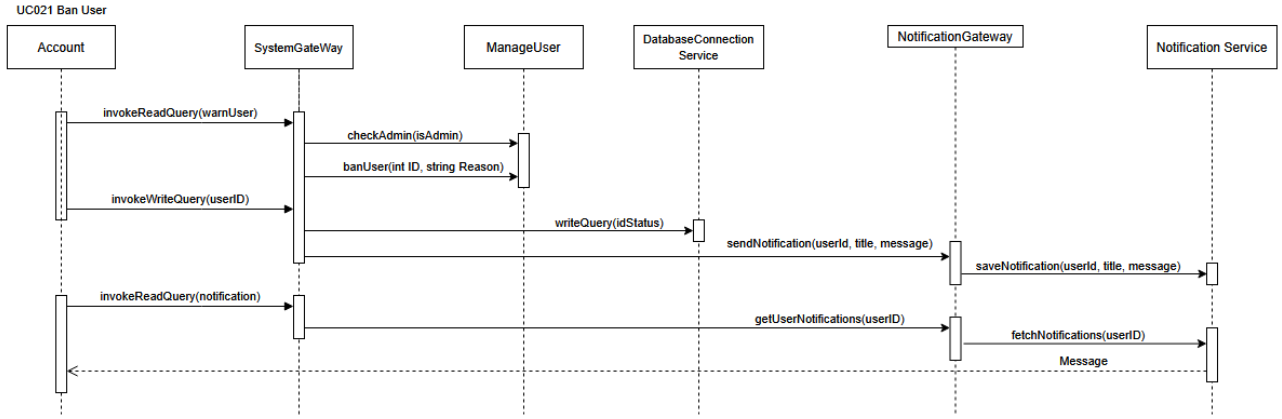
3.3.20 UC020 Suspend User Sequence Diagram



Author: Chayathon Chaiyasak

This diagram shows the flow of the UC020 Suspend user use case. The admin selects a user to suspend. The request is sent through the SystemGateway, which first checks admin permissions. If the user is an admin, the system sends the suspension request (with reason) to ManageUser. This triggers an update in the database to change the user's status. After the suspension is recorded, a notification is sent to the user via the NotificationGateway, which stores it using the Notification Service.The system then fetches and displays updated notifications for the suspended user.

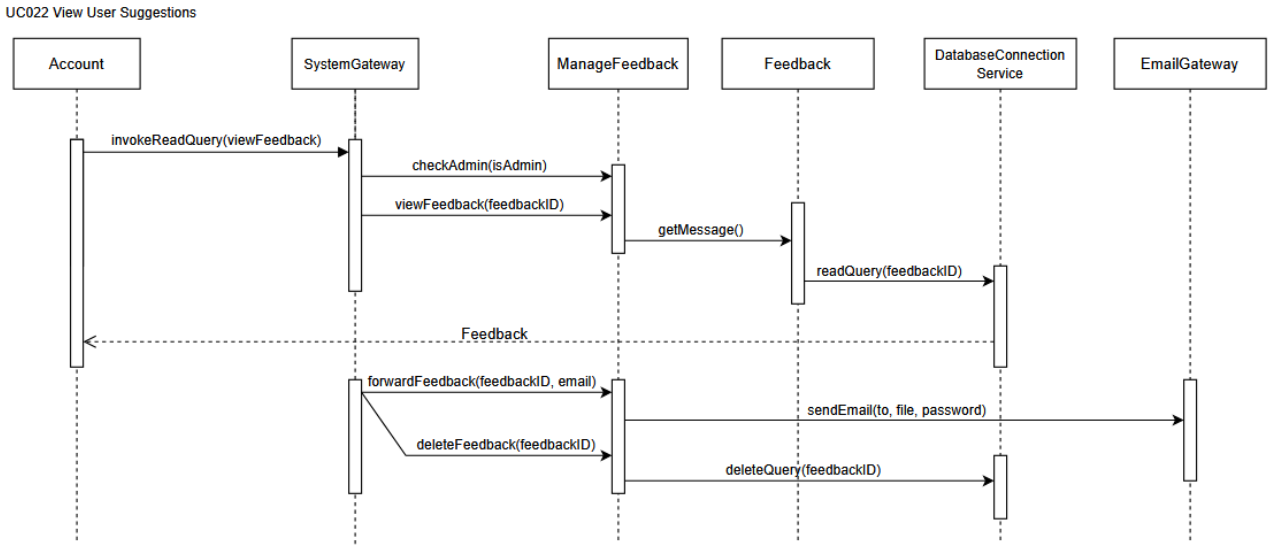
3.3.21 UC021 Ban User Sequence Diagram



Author: Pattarapong Tiemtan

This diagram shows the flow of the UC021 Ban User use case. First, the system if the account is admin or not. Second, the account with admin type inputs the user ID and the reason for the ban. Third, the database is called to ban the user. After that, the system writes the message and prepares to be sent. Finally, the Notification Service sends the notification to the said user.

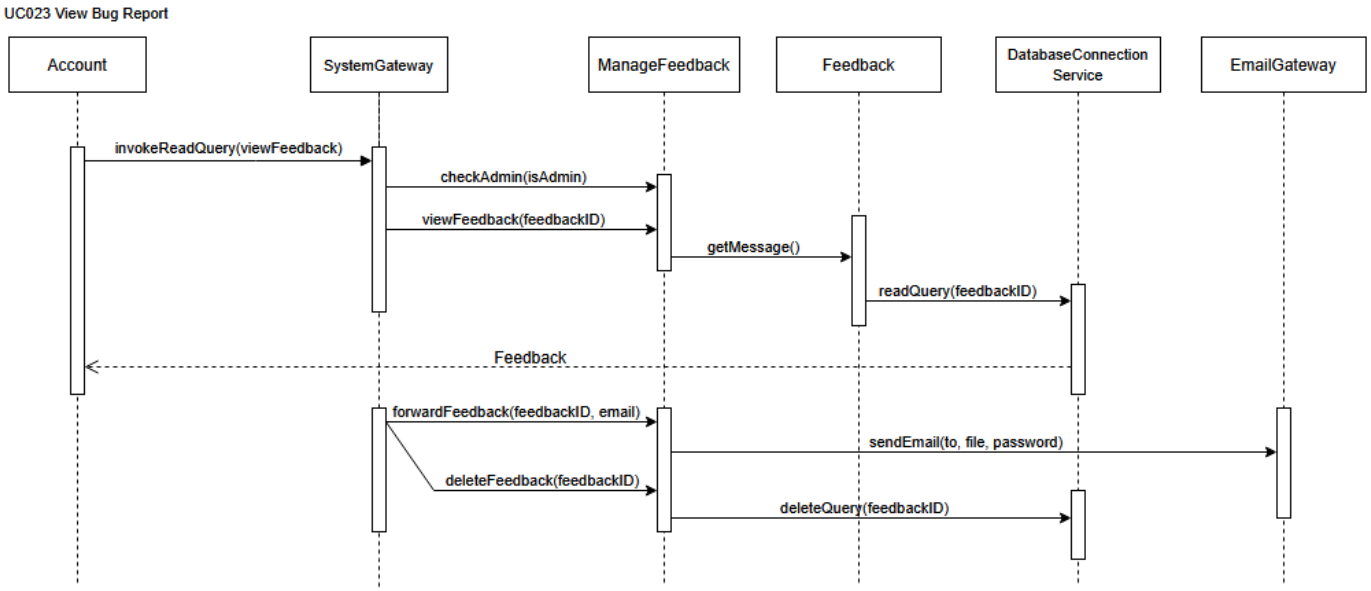
3.3.22 UC022 View User Suggestions Sequence Diagram



Author: Pattarapong Tiemtan

This diagram shows the flow of the UC022 View User Suggestion use case. In this user case, enable the admin to view, delete or forward the user suggestion. First, the account must be admin in order to access the ManageFeedback function. Second, the SystemGateway will evoke the DatabaseConnection Service so that it can fetch the feedback from the feedbackID back to the admin. Finally, after the admin viewed the feedway, they can choose two choices: to forward or delete. In the forward option, the ManageFeedback will automatically generate the report and send it to the related department by email with EmailGateway. In the delete option, the system will delete the feedback by using feedbackID to identify the feedback in the database.

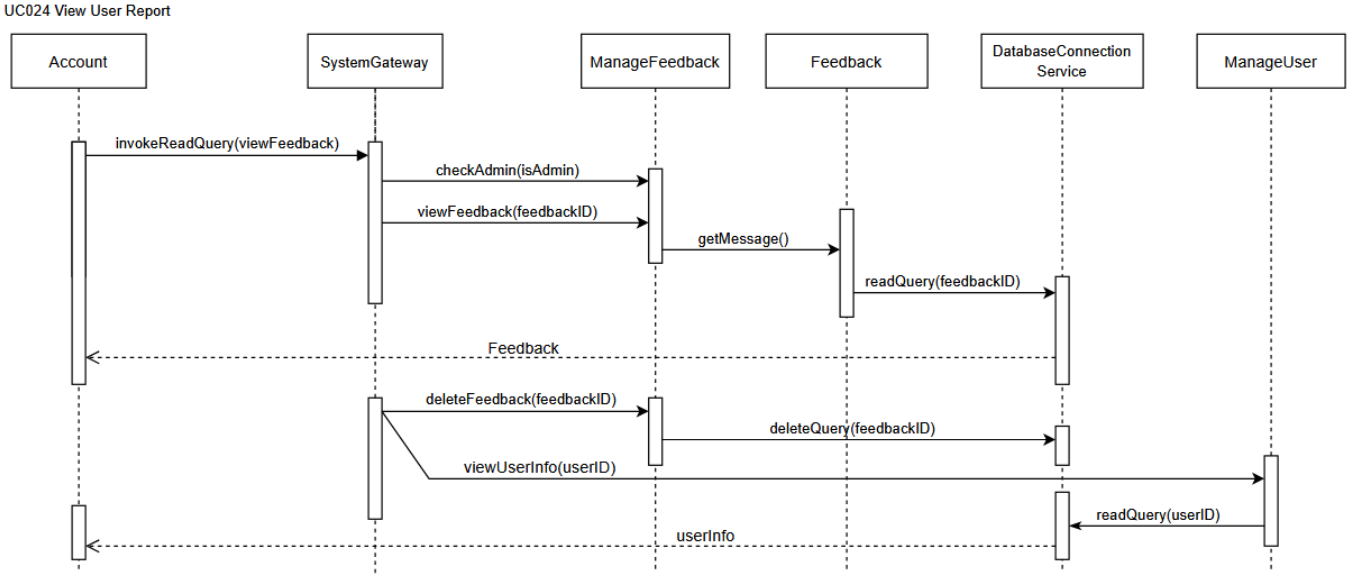
3.3.23 UC023 View Bug Report Sequence Diagram



Author: Pattarapong Tiemtan

This diagram shows the flow of the UC023 View Bug Report use case. Same as the previous user case, the account must be admin. In this user case, the admin will be able to view the bug report that was sent by the user. To add up, the admin can choose to delete the bug report or forward it to the developer in order to fix the bug.

3.3.24 UC024 View User Report Sequence Diagram

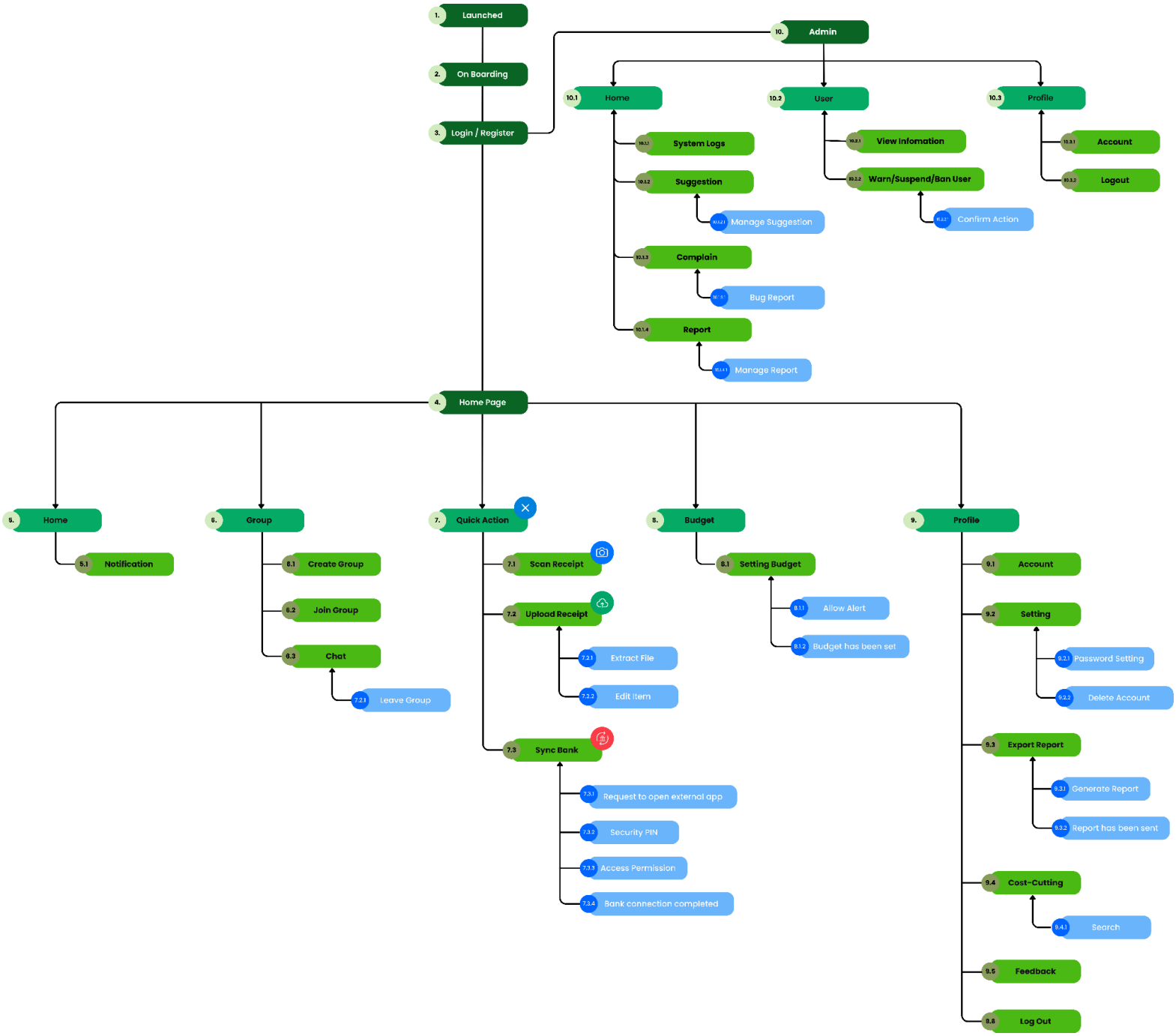


Author: Pattarapong Tiemtan

This diagram shows the flow of the UC024 View User Report use case. To access the functionality, the user must be admin. The admin can view the user report via the ManageFeedback. The admin then has two options: delete and accept the report. In the delete option, the admin can delete the report from the database. In the accept option, the report will be saved with the userID and change page to user management for further operation.

4 Human Interface Design

4.1 System Navigation Design



https://drive.google.com/file/d/1bDXFuh0KB-wHW2xEEZGFhO9cJTNMqPBM/view?usp=drive_link

4.2 Screen Design

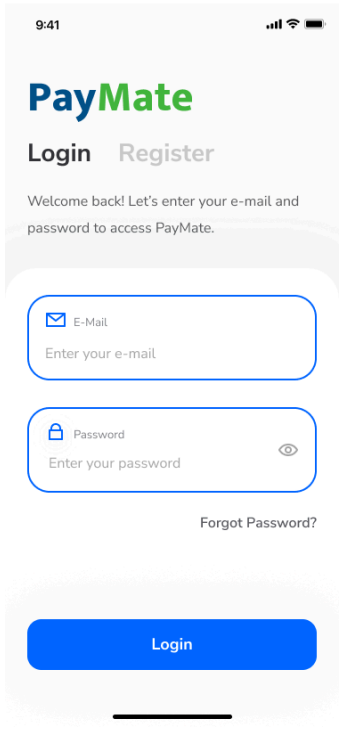
Display screenshots showing the interface from the user's perspective. Show the screen mock-up design.

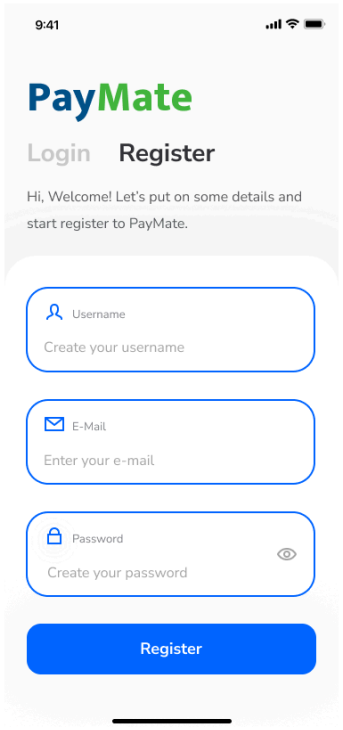
Onboarding



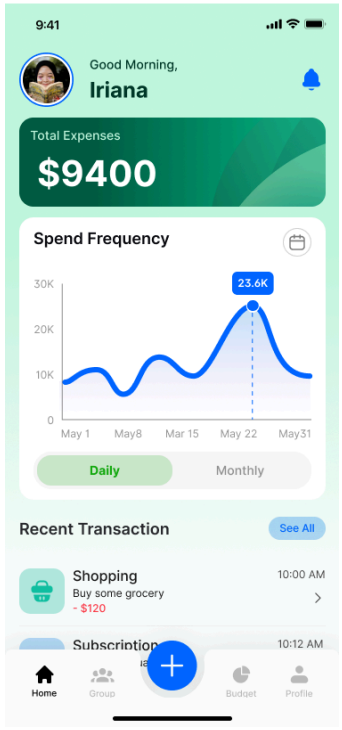


Login & Register



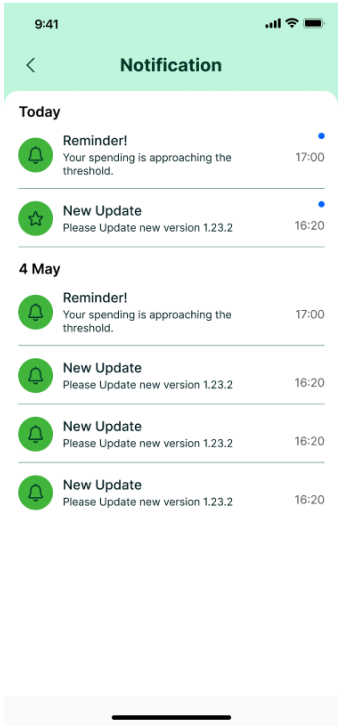


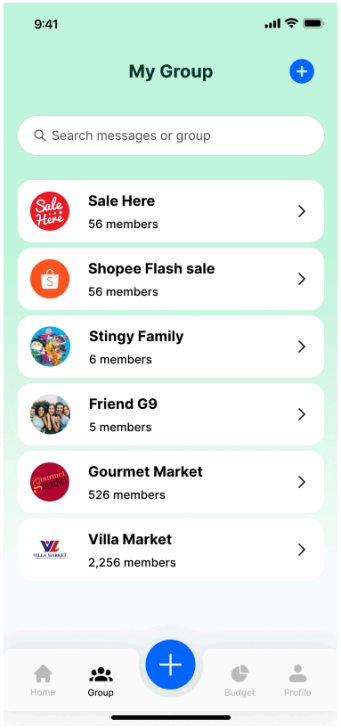
Home

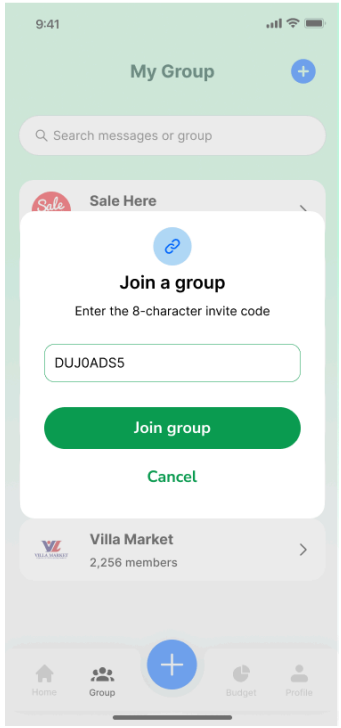


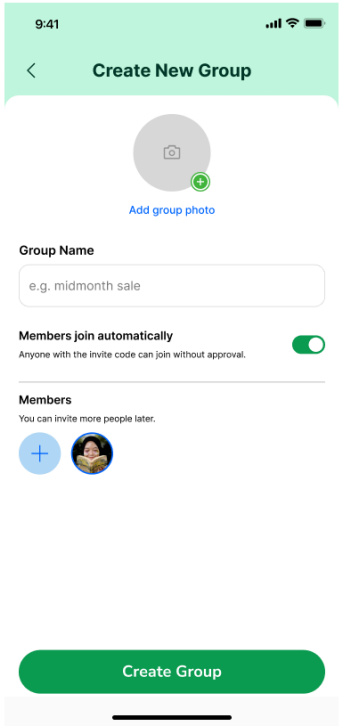
Home | **Group**

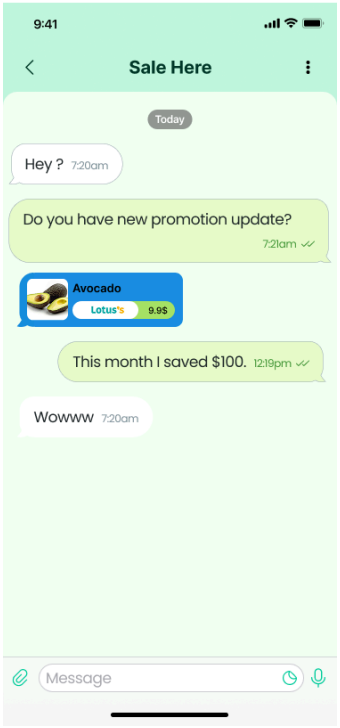
Notification | **Join** | **Create** | **Chat**





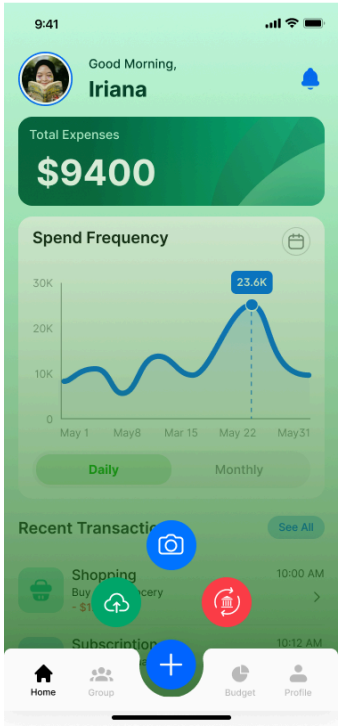






Quick Action

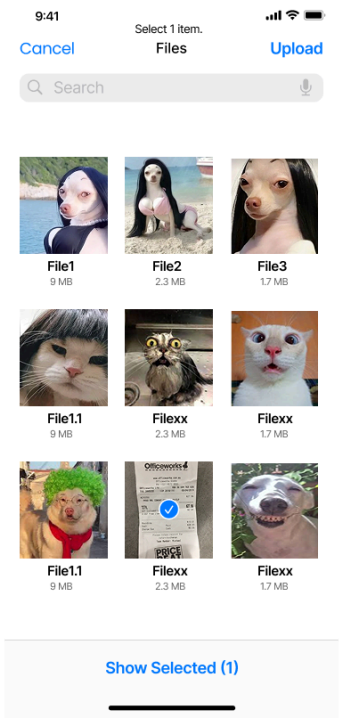
Scan



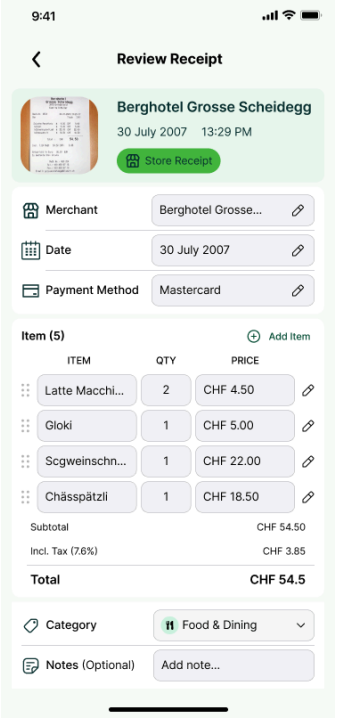




Upload

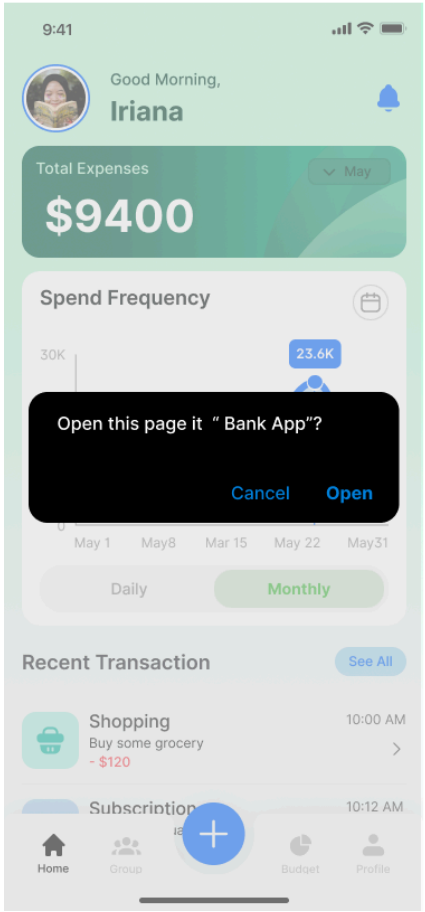


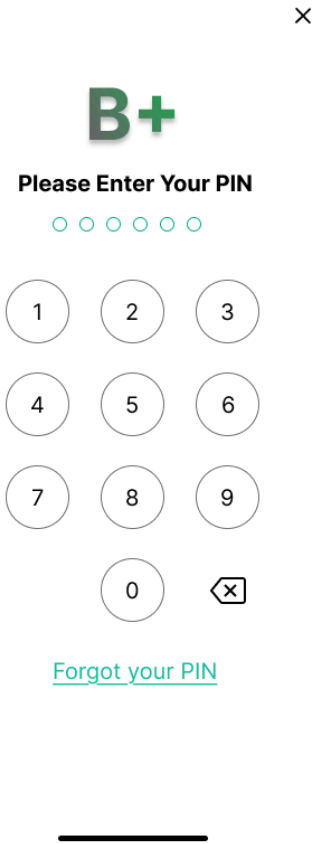
Edit Item

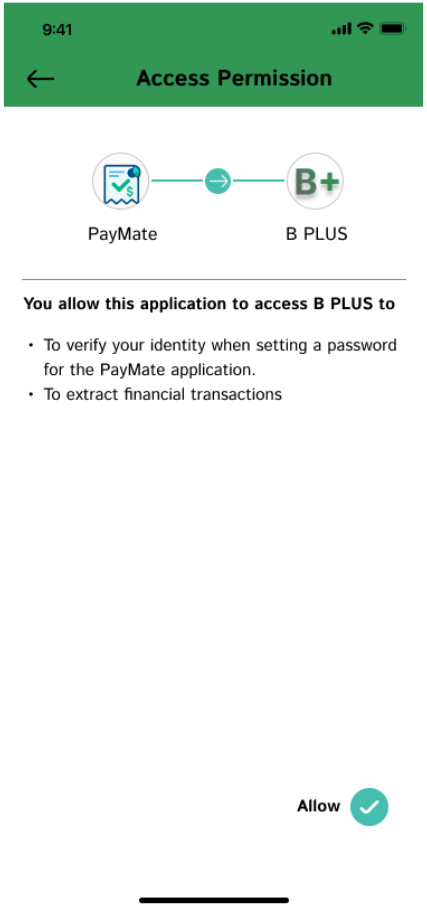


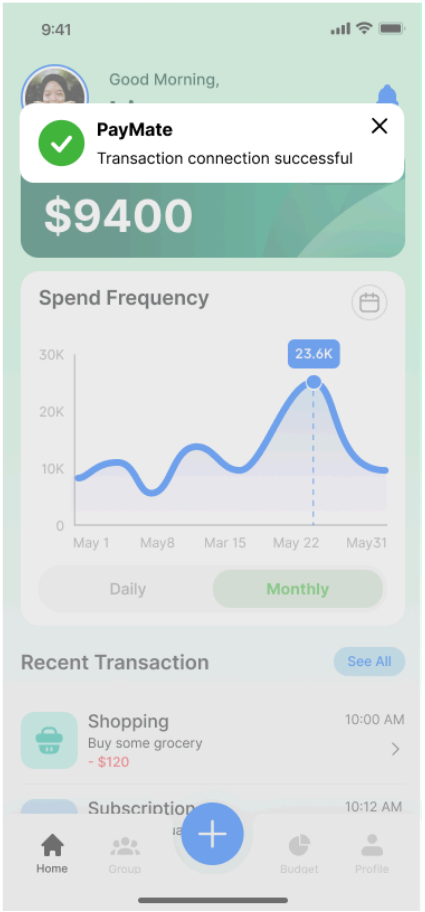
Quick Action

SyncBank



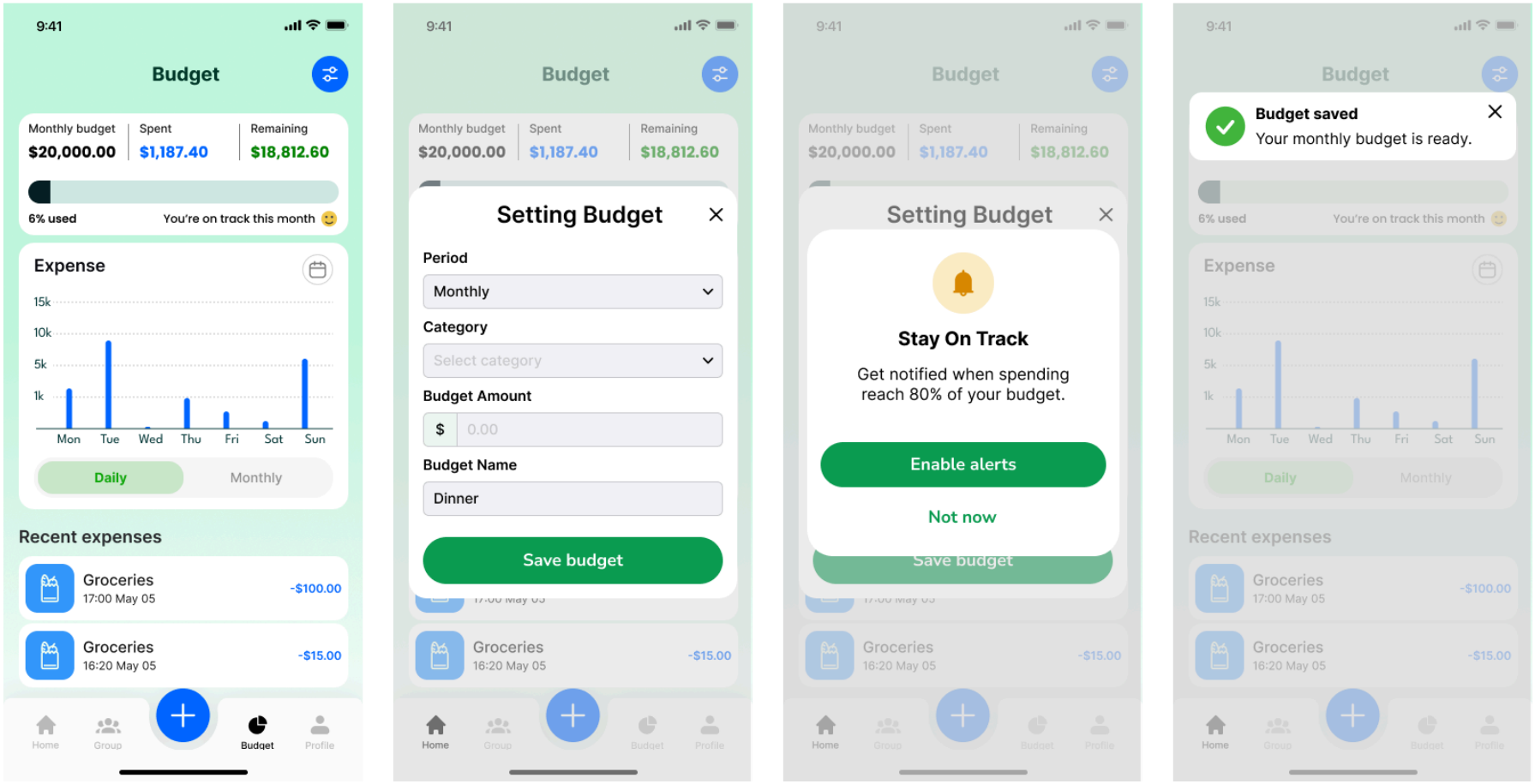






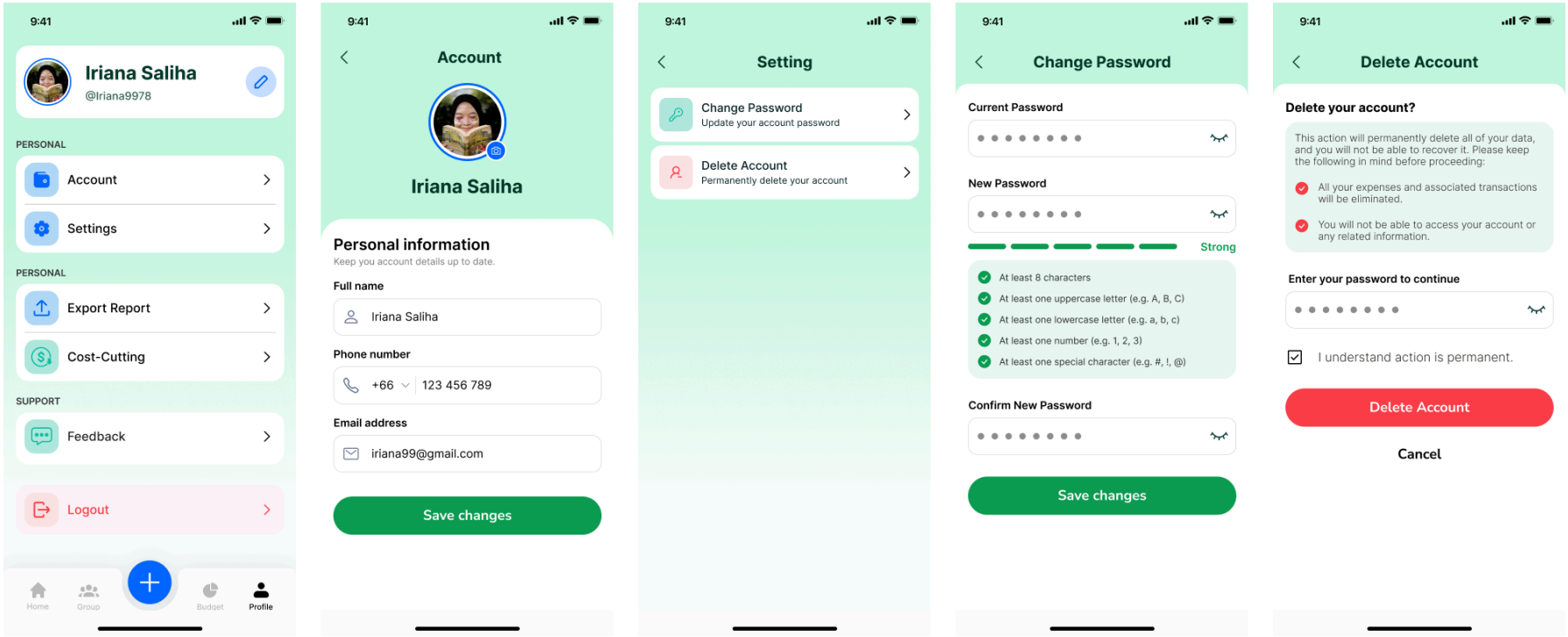
Budget

Budget Setting



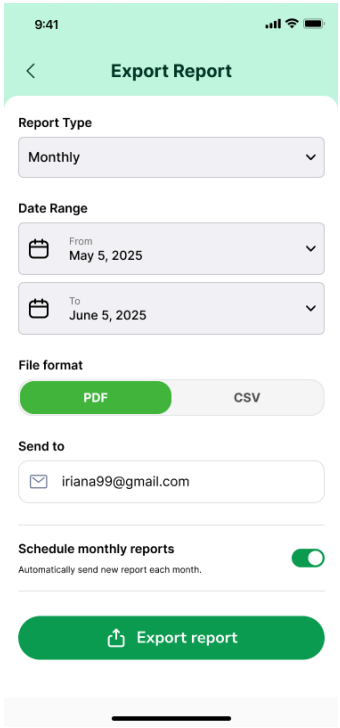
Profile

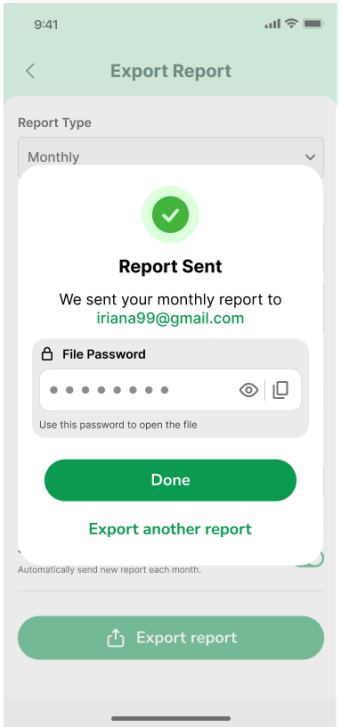
Account **Setting**



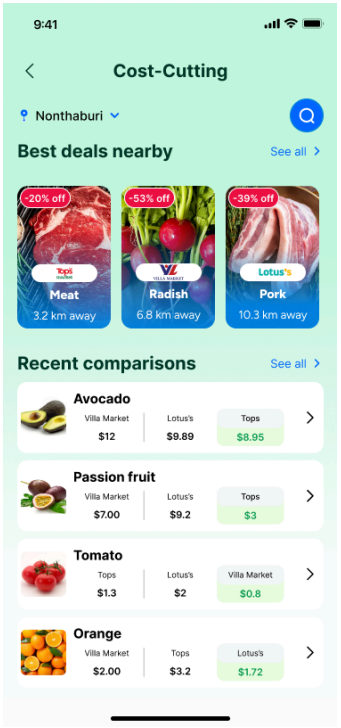
Profile

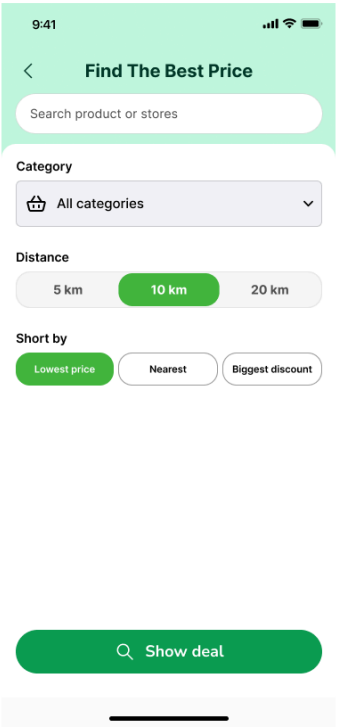
Export Report



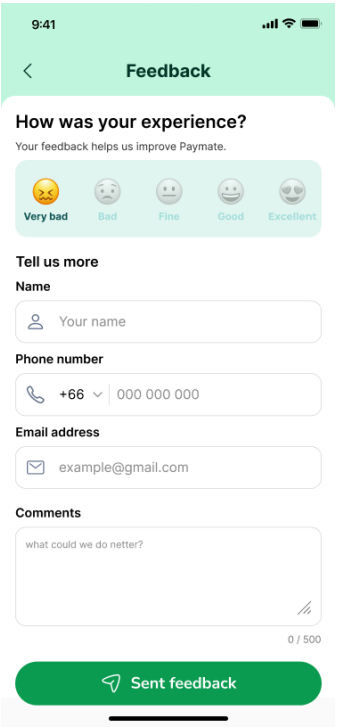


Cost-cutting



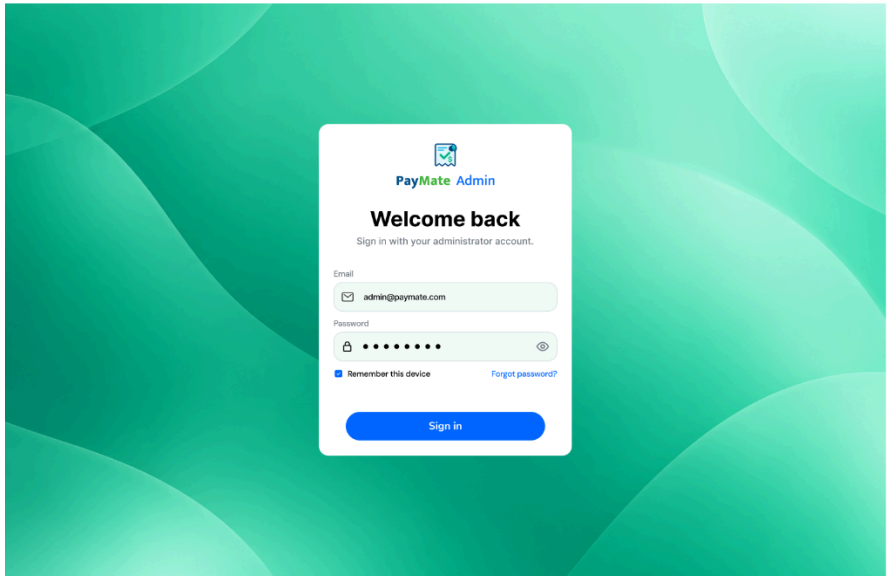


Feedback

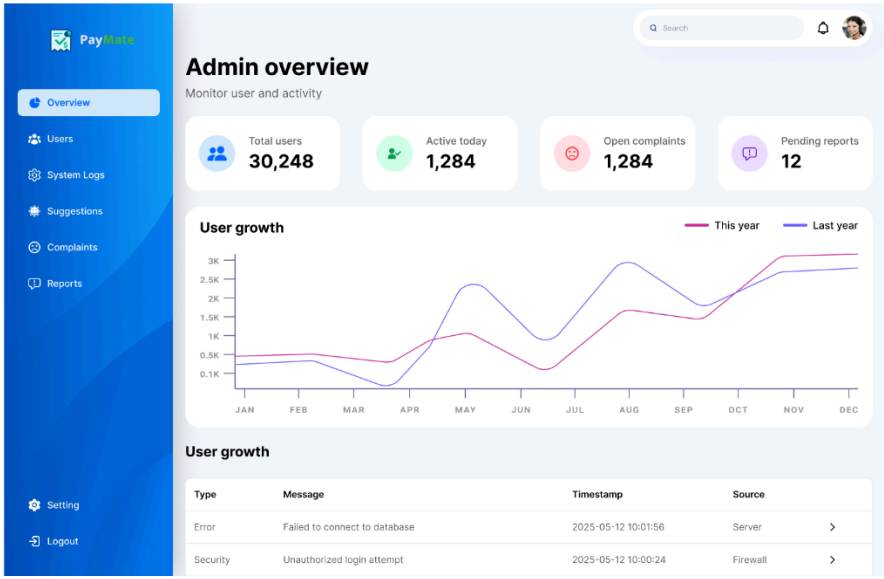


Admin

Login

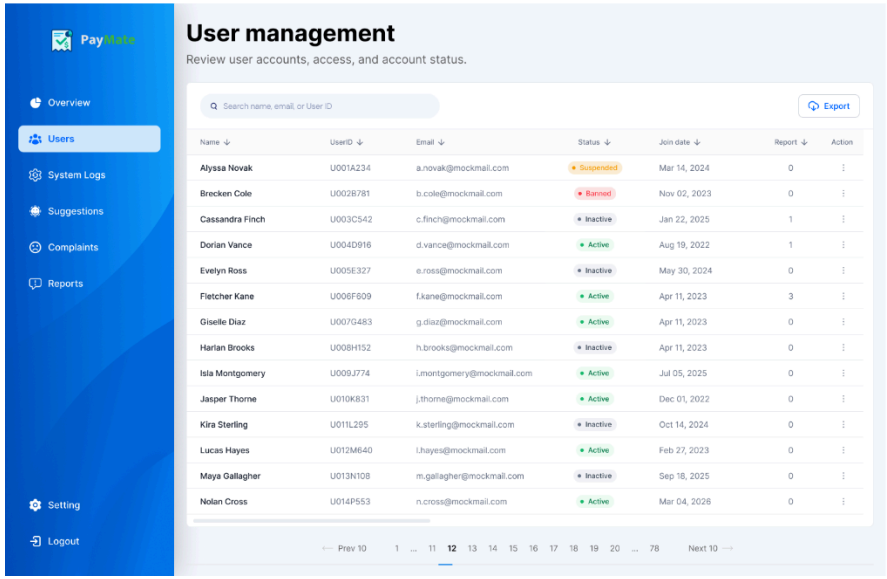


Overview

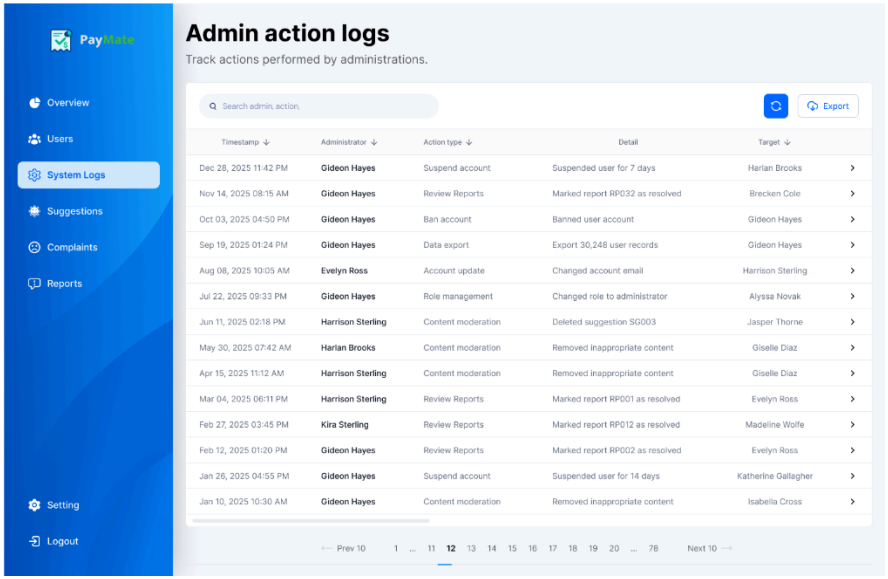


Admin

User management

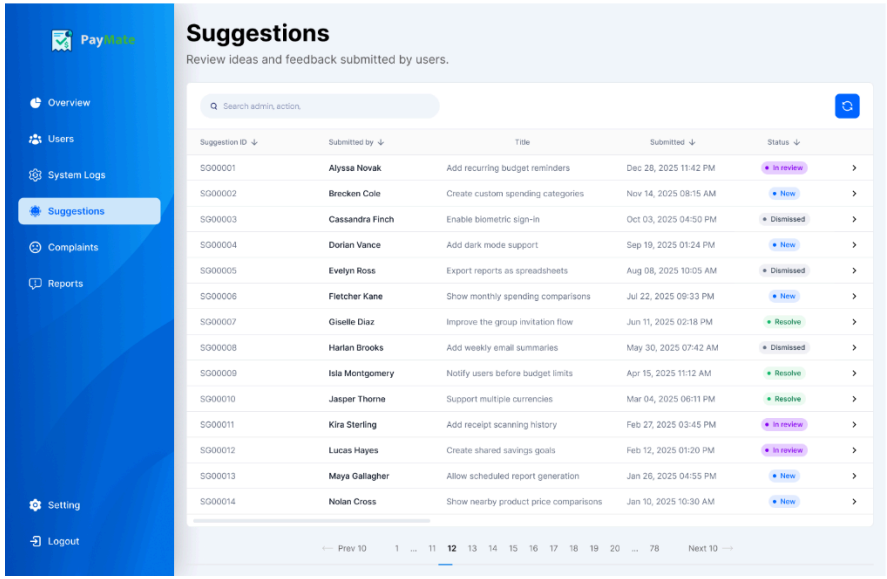


System logs

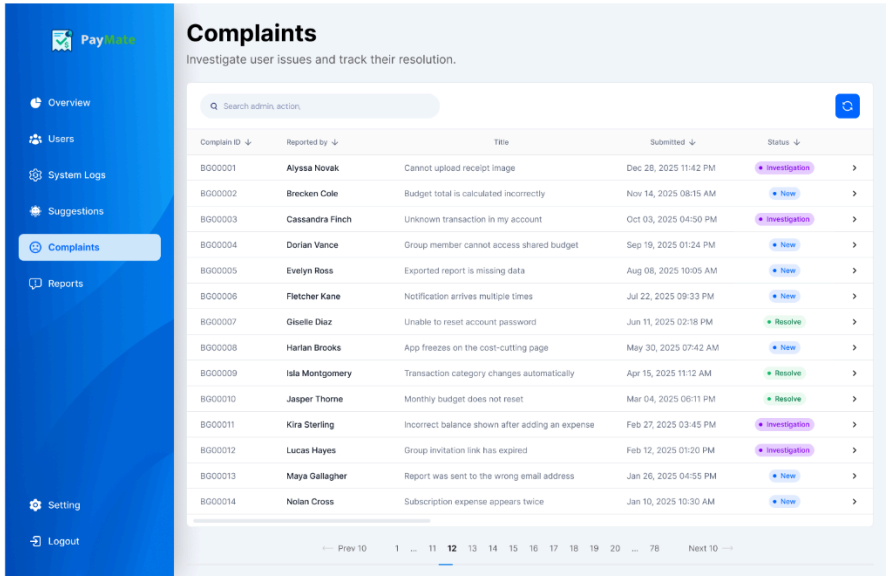


Admin

Suggestions

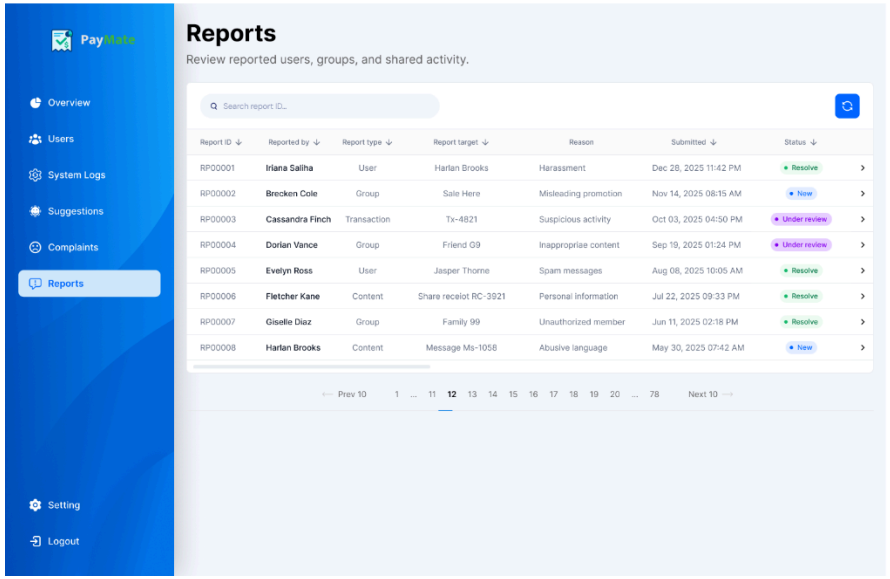


Complaints

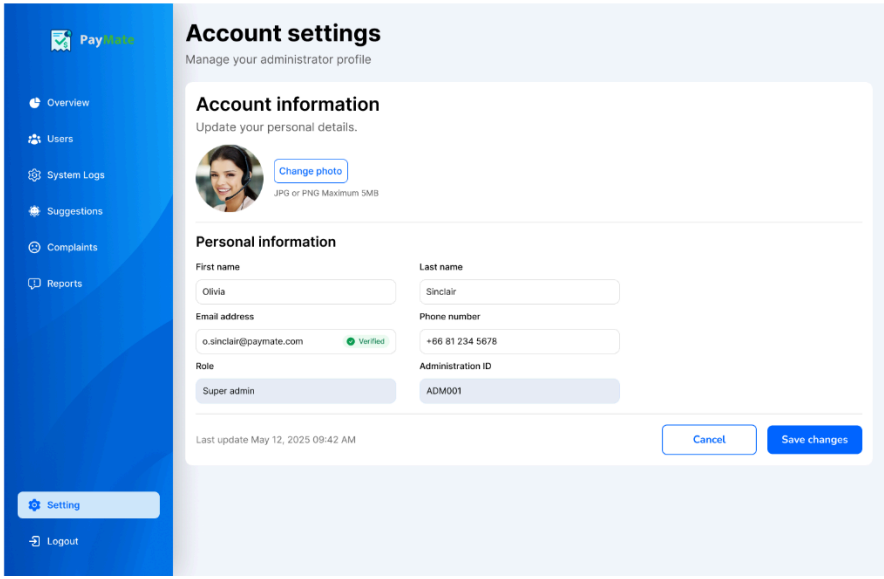


Admin

Reports



Account settings



5 Requirement Traceability

Show Requirement Traceability Matric in this section by adding Screen ID in the excel file.

System Requirements			
Functional Requirement	Quality Requirement	Use Case	Use Case Description
FR001: The system shall allow the user to select between "bill splitting" or "bill customization."	QR001: The system shall respond to selection inputs within 1 second.	UC005	UD002
FR002: The system shall allow the user to edit or remove specific bill items after scanning.	QR002: The system shall process item selection and updates within 3 seconds.	UC005	UC003
FR001: The system shall allow the user to select between "bill splitting" or "bill customization."	QR001: The system shall respond to selection inputs within 1 second.	UC005	UD002
FR003: The system shall allow the user to select between "splitting" (Divides the total bill amount equally among a specified number of people) or "custom amount" (Allows the user to input a custom value for the amount).	QR003: The system shall complete bill splitting within 2 seconds for any valid input (i.e., total bill amount and number of people). QR004: The system shall validate that the custom amount input does not exceed the total bill amount before proceeding with the transaction.	UC005	UD005
FR004: The system shall automatically extract items, store name, payment method, date, and time from different types of bills.	QR005: The system shall complete data extraction within 3 seconds.	UC005	UD005
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR006: The system shall allow the user to search for cost-cutting alternatives on the web relevant to the user's interests.	QR007: The system shall complete web scraping within 10 seconds, with results ranked by relevance.	UC007	UD007

FR007: The system shall provide suggestions and comparisons for lower-cost alternatives.	QR008: The system shall update cost-saving suggestions at least once per day, reflecting real-time availability.	UC007	UD007
FR008: The system shall provide notifications for new suggestions.	QR009: The system shall deliver notifications within 5 seconds of new cost-cutting suggestions being found.	UC007	UD007
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR009: The system shall display the total expense of the month.	QR010: The system shall load the dashboard expense summary within 2 seconds for optimal user experience.	UC005	UD005
FR010: The system shall display the expense trend over a selected time period (weekly/monthly).	QR011: The system shall complete trend calculations within 3 seconds for datasets under 12 months.	UC005	UD005
FR011: The system shall allow the user to sort displayed receipts (e.g., by time, by amount).	QR012: The system shall complete sorting operations within 1 second for datasets under 1000 receipts.	UC005	UD005
FR005: The system shall automatically categorize expenses into predefined categories (e.g., Groceries, Utilities, Rent, Entertainment) based on receipt details.	QR006: The system shall complete data categorization within 1 second.	UC005	UD005
FR012: The system shall allow users to integrate with bank APIs to sync transactions.	QR013: The system shall sync bank transactions within 5 seconds of new transactions being detected. QR014: The system shall encrypt all stored receipts using AES-256 to ensure data confidentiality.	UC006	UD006
FR013: The system shall allow the user to set and edit customizable budget periods.	QR015: The system shall persist budget settings and reflect changes immediately.	UC008	UD008
FR014: The system shall allow the user to set and edit a budget amount.	QR016: The system shall prevent negative or invalid values in budget input.	UC008	UD008
FR015: The system shall allow the user to track budget progress.	QR017: The system shall calculate and update budget progress when user inputs new receipts.	UC008	UD008
FR016: The system shall store scanned receipt images along with extracted data in categories.	QR014: The system shall encrypt all stored receipts using AES-256 to ensure data confidentiality.	UC005	UD005
FR017: The system shall allow the user to filter receipts by category.	QR018: Querying operations shall be completed in under 2 seconds.	UC005	UD005
FR018: The system shall allow the user to manually delete records.	QR014: The system shall encrypt all stored receipts using AES-256 to ensure data confidentiality.	UC005	UD005

FR019: The system shall generate reports for timeframes of 1 month, 6 months, and 1 year.	QR019: The system shall complete report generation within: 5 seconds for a 1-month timeframe, 10 seconds for a 6-month timeframe, 15 seconds for a 1-year timeframe	UC009, UC010	UD009, UD010
FR020: The system shall allow the user to manually generate reports.	QR020: The system shall generate manual reports within 10 seconds of being prompted.	UC009	UD009
FR021: The system shall allow the user to automatically generate reports.	QR021: The system shall generate automated reports within 1 minute of the scheduled time.	UC010	UD010
FR022: The system shall send the report to the user's email.	QR022: The system shall deliver emails within 30 seconds of report generation.	UC009, UC010	UD090, UD010
FR023: The system shall allow the user to upload receipts via camera capture.	QR023: The system shall detect blurry images and prompt the user to retake the photo before processing.	UC003	UD003
FR024: The system shall allow the user to upload receipts via file upload.	QR024: The system shall process uploaded receipt files within 3 seconds, with a maximum file size of 50MB. QR025: The system shall accept only PDF, JPEG, and PNG file types.	UC004	UD004
FR025: The system shall process uploaded receipts using OCR.	QR017: The system shall process OCR operations within 5 seconds maximum.	UC003, UC004	UD003, UD004
FR026: The system shall allow the user to create a group.	QR026: The system shall support up to 10 members per group.	UC011	UD011
FR027: The system shall allow the user to share an invitation link to join a group.	QR027: The system shall ensure that invitation links are valid for 24 hours and regenerated regularly.	UC011, UC012	UD011, UD012
FR028: The system shall allow the user to log shared expenses in a group.	QR028: The system shall sync group changes in real-time across all user devices.	UC014	UD014
FR029: The system shall allow the user to leave a group.	QR029: The system shall immediately remove access when a user leaves a group but retain past records.	UC013	UD013
FR030: The system shall provide notifications when new records are added by other users.	QR030: The system shall maintain detailed activity logs to track changes made by group members.	UC014	UD014
FR031: The system shall allow the user to log in using existing credentials.	QR031: The login process should take no longer than 2 seconds.	UC001	UD001
FR032: The system shall allow the user to register as a new user.	QR032: The registration form should have clear instructions and input validation to ensure users provide correct information.	UC002	UD002

FR033: The system shall allow the user to create and submit general suggestions.	QR033: The suggestion submission form should confirm successful submission or provide an error message within 3 seconds of submission.	UC015	UD015
FR034: The system shall allow the user to create and submit bug reports.	QR034: The bug reporting feature should be available 24/7 with 99.9% per year uptime.	UC015	UD015
FR035: The system shall allow the user to report specific users.	QR035: Reports about users must be anonymous to the public.	UC015	UD015
FR036: The system shall allow the user to search for other users.	QR036: User searching should take no longer than 2 seconds.	UC017	UD017
FR037: The system shall allow the user to view another user's profile, information, and activities.	QR037: Profile details should be fetched within 3 seconds.	UC017	UD017
FR038: The system shall allow the admin to fetch all logs.	QR038: Log retrieval should be optimized for searches spanning up to 6 months.	UC018	UD018
FR039: The system shall allow the admin to view all logs of the application.	QR039: The system shall display real-time system logs.	UC018	UD018
FR040: The system shall allow the admin to select the type of logs to view.	QR040: Log filtering should take under 2 seconds.	UC018	UD018
FR041: The system shall allow the admin to fetch user information.	QR041: The system shall ensure real-time synchronization of user management with the server.	UC017	UD017
FR042: The system shall allow the admin to send warnings to a user.	QR042: Warnings should be delivered within 5 seconds via notification/email.	UC019	UD019
FR043: The system shall allow the admin to suspend a user.	QR043: Suspension should immediately restrict login access.	UC020	UD020
FR044: The system shall allow the admin to ban a user.	QR044: Banning should permanently disable all user access and retain some data for compliance.	UC021	UD021
FR045: The system shall allow the admin to fetch user suggestions, bug reports, or user reports.	QR045: Reports should be fetched within 3 seconds.	UC022, UC023, UC024	UD022, UD023, UD024
FR046: The system shall allow the admin to view user suggestions, bug reports, or user reports.	QR046: Displaying reports should allow sorting by date or category.	UC022, UC023, UC024	UD022, UD023, UD024
FR047: The system shall allow the admin to delete user suggestions, bug reports, or user reports.	QR047: Deletion should be permanent and require admin confirmation.	UC022, UC023, UC024	UD022, UD023, UD024

FR048: The system shall allow the admin to forward user suggestions, bug reports, or user reports to the related department.	QR048: Forwarding should automatically generate a tracking ID for follow-up.	UC022, UC023, UC024	UD022, UD023, UD024
FR049: The system shall automatically generate email reports.	QR049: Email generation should take under 5 seconds for reports under 1000 entries.	UC022, UC023, UC024	UD022, UD023, UD024
FR031: The system shall allow the admin to log in using existing credentials.	QR031: The login process should take no longer than 2 seconds.	UC016	UD016

Requirement Traceability Sheet Original File:
https://docs.google.com/spreadsheets/d/1g6-TCD6_uJHf1Wn2zgQoujm4KaemlxcxZQzLZg4x3is/edit?usp=sharing



Appendix A – Data Dictionary

Data Element	Description	Data Type	Size	Constraints
UserID	Unique identifier for user	INT	20	Primary Key, Not Null, Unique
Name	Full name of the user	VARCHAR	100	Not Null
Email	Unique email address	VARCHAR	150	Not Null, Unique
Password_hash	Hashed password	VARCHAR	255	Not Null
Created_at	Account creation timestamp	DATETIME	-	Not Null
Login_method	Method used to log in (e.g., Email, Google)	VARCHAR	50	Not Null
ReceiptID	Unique receipt identifier	INT	20	Primary Key, Not Null, Unique
ImagePath	Path to the receipt image	TEXT		Not Null
VendorName	Vendor/store name	VARCHAR	100	Nullable
PurchaseDate	Date of the transaction	DATE		Not Null
OCR_Text	Raw OCR output for future validation	TEXT		Nullable
CategoryID	Unique category identifier	INT	20	Primary Key, Not Null, Unique
GroupName	Name of the group(e.g., Electricity, Water, Internet, etc.)	VARCHAR	100	Not Null, Unique
GroupID	Group identifier	INT	20	Primary Key, Not Null, Unique
GroupName	Name of the group/household	INT	100	Not Null
Role	Role (e.g., admin, member)	VARCHAR	50	Not Null
AccountID	Unique bank account identifier	INT	20	Primary Key, Not Null, Unique
BankName	Name of the bank	VARCHAR	100	Not Null
AccountNumber	Masked or tokenized account number	VARCHAR	50	Not Null
TransactionID	Unique transaction ID	INT	20	Primary Key, Not Null, Unique
AccountID	Unique bank account identifier	INT	20	Foreign Key → Account(AccountID)
Amount	Transaction amount	DECIMAL	100	Not Null
Date	Transaction date	DATE		Not Null
Description	Description of the transaction	VARCHAR	255	Nullable
Linked_Receipt_ID	Associated receipt (if linked)	INT	20	Foreign Key → Receipt(ReceiptID), Nullable
ReportID	Unique report identifier	VARCHAR	20	Primary Key, Not Null, Unique

ReportType	Type (e.g., "PDF", "CSV", "PNG")	VARCHAR	50	Not Null
PasswordProtected	Boolean flag for password protection	BOOLEAN		Default:false
CreatedDate	Time to generate report	DATE		Not Null
BudgetID	Unique budget identifier	INT	20	Primary Key, Not Null, Unique
LimitAmount	The max budget set for this category	DECIMAL		Not Null
StartDate	Start date of the budget period (e.g., start of the month)	DATE		Not Null
EndDate	End date of the budget period (e.g., end of the month)	DATE		Not Null
ItemID	Unique line item ID	INT	20	Primary Key, Not Null, Unique
ItemName	Name or description of the item (e.g., "Electricity Unit")	VARCHAR	100	Not Null
Quantity	Quantity of the item (e.g., 2, 3.5 units)	INT		Not Null
UnitPrice	Price per unit	DECIMAL	(10,2)	Not Null
SuggestionID	Unique suggestion ID	INT	20	Primary Key, Not Null, Unique
Description	Suggestion text (e.g., "Switch to XYZ provider to save 20%")	TEXT		Not Null
Savings_Amount	Estimated savings if the suggestion is followed	DECIMAL		Nullable
NotificationID	Unique notification ID	INT	20	Primary Key, Not Null, Unique
Message	The content of the notification (e.g., new suggestion).	VARCHAR	100	Not Null
Notification_type	Type of notification (e.g., "Reminder", "Suggestion", "Alert").	VARCHAR	50	Not Null
Is_Read	Flag to indicate whether the notification has been read.	BOOLEAN		Default:false

Appendix B - Group Log

Date	Activity	Members Involved
27/1/2025	● First Group Meeting ● Project Proposal Slide Completion	*Everyone*
19/2/2025	● Requirement Traceability interviews ● Requirement Traceability sheet ● First draft of BPMN diagram	*Everyone*
25/2/2025	● Continue working on BPMN diagram ● Continue working on Requirement Traceability Sheet	*Everyone*
27/2/2025	● Use Case Synthesis on Requirement Traceability Sheet ● Beginning of work on SRS report	*Everyone*
1/3/2025	● Completion of BPMN diagram ● Completion of Requirement Traceability sheet ● Continue working on SRS report	*Everyone*
2/3/2025	● Continue working on SRS report	*Everyone*
3/3/2025	● Completion of SRS report	*Everyone*
29/3/2025	● Second Group Meeting ● First draft of Class diagram and Sequence diagram	*Everyone*
5/4/2025	● Continue working on Class diagram ● Continue working on Sequence diagram	*Everyone*
9/4/2025	● Completion of Class diagram ● Completion of Sequence diagram	*Everyone*
12/4/2025	● Third Group Meeting ● First draft of UI	*Everyone*
19/4/2025	● Completion of UI Design ● Beginning of work on SDS report	*Everyone*
12/5/2025	● Completion of SDS report	*Everyone*